

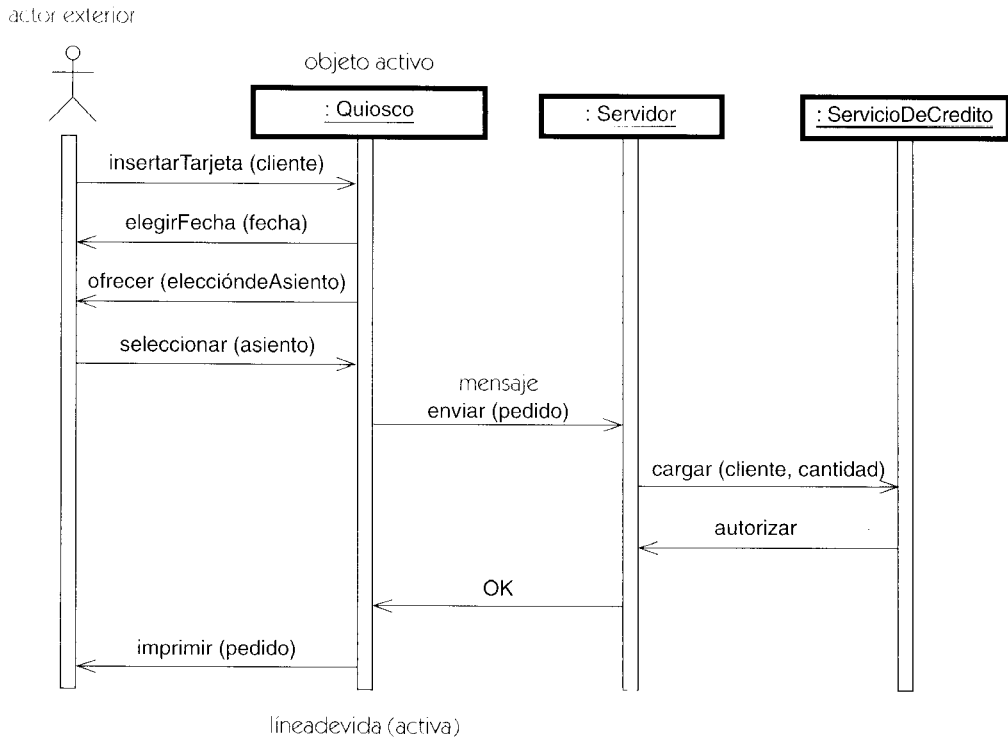


Figura 8.1 Diagrama de secuencia

tal muestra los roles de clasificador que representan objetos individuales en la colaboración. Cada rol de clasificador se representa mediante una columna vertical—línea de vida. Durante el tiempo que existe un objeto, el rol se muestra por una línea discontinua. Durante el tiempo que dura una activación de un procedimiento en el objeto, la línea de vida se dibuja como una línea doble.

Se muestra un mensaje como una flecha desde la línea de vida de un objeto a la del otro. Las flechas se organizan en el diagrama en orden cronológico hacia abajo.

La Figura 8.1 muestra un diagrama de secuencia típico con mensajes asíncronos.

Activación

Una activación es la ejecución de un procedimiento, incluyendo el tiempo que espera a los procedimientos anidados para ejecutarse. Se representa por una línea doble que sustituye la parte de la línea de vida en un diagrama de secuencia. Una llamada se representa por una flecha que apunta a la parte superior de la activación iniciada por la llamada. Ocurre una llamada recursiva cuando el control vuelve a entrar en una operación en un objeto, pero la segunda llamada es una activación separada de la primera. La recursión o una llamada anidada a otra operación en el mismo objeto, se representa en un diagrama de secuencia apilando las líneas de activación. La Figura 8.2 muestra un diagrama de secuencia, con el flujo de objeto de procedimientos, incluyendo una llamada recursiva y la creación de un objeto durante el cómputo.

Un objeto activo es un objeto que contiene la raíz de una pila de activaciones. Cada objeto activo tiene su propio hilo de control dirigido por eventos que se ejecuta en paralelo a otros ob-

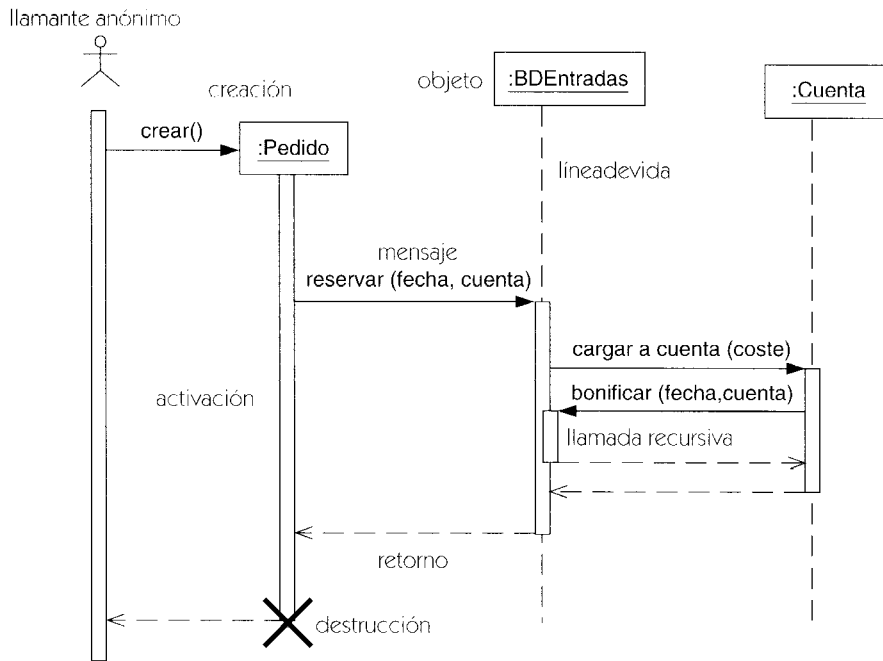


Figura 8.2 Cuando dos hilos van juntos en un solo objeto

jetos activos. Los objetos que son llamados por un objeto activo son objetos pasivos; reciben el control solamente cuando son llamados, y lo ceden cuando retornan.

Si varios hilos de control concurrentes tienen sus propios flujos de procedimientos de control usando llamadas anidadas, los diversos hilos deben distinguirse usando los nombres de los hilos, colores, u otros medios para evitar la confusión cuando dos hilos se unen en un único objeto (para una sincronización, por ejemplo). Generalmente, es mejor no mezclar llamadas a procedimientos con señales en un solo diagrama.

Diagrama de colaboración

Un diagrama de colaboración es un diagrama de clases que contiene roles de clasificador y roles de asociación en lugar de sólo clasificadores y asociaciones. Los roles de clasificador y los roles de asociación describen la configuración de los objetos y de los enlaces que pueden ocurrir cuando se ejecuta una instancia de la colaboración. Cuando se instancia la colaboración, los objetos están ligados a los roles de clasificador y los enlaces están ligados a los roles de asociación. El rol de asociación también puede ser desempeñado por varios tipos de enlaces temporales, tales como argumentos de procedimiento o variables locales del procedimiento. Los símbolos de enlace pueden llevar estereotipos para indicar enlaces temporales («**parameter**» o «**local**») o llamadas al mismo objeto («**self**»).

Solamente se representan los objetos que están implicados en la colaboración, aunque puede haber otros en el sistema completo. Es decir, un diagrama de colaboración modela los objetos y los enlaces implicados en la implementación de una interacción y no hace caso de las otras. La Figura 8.3 muestra un diagrama de colaboración.

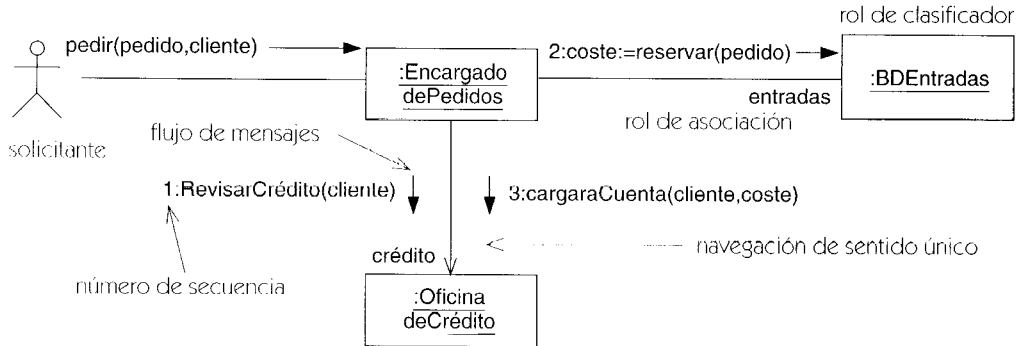


Figura 8.3 Diagrama de colaboración

Es útil marcar los objetos en cuatro grupos: los que existen con la interacción entera; los creados durante la interacción (restricción **{new}**); los destruidos durante la interacción (restricción **{destroyed}**); y los que se crean y se destruyen durante la interacción (restricción **{transient}**). Durante el diseño, se puede empezar mostrando los objetos y los enlaces disponibles al comienzo de una operación y después decidir cómo el control puede fluir a los objetos correctos dentro del gráfico para implementar la operación.

Aunque las colaboraciones muestran directamente la implementación de una operación, pueden también mostrar la realización de una clase entera. En este uso, muestran el contexto necesario para implementar *todas* las operaciones de una clase. Esto permite que el modelador vea los roles múltiples que los objetos pueden desempeñar en varias operaciones. Esta vista puede ser construida tomando la unión de todas las colaboraciones necesarias para describir todas las operaciones del objeto.

Mensajes. Los mensajes se muestran como flechas etiquetadas unidas a los enlaces. Cada mensaje tiene un número de secuencia, una lista opcional de mensajes precedentes, una condición opcional de guarda, un nombre y lista de argumentos, y un nombre de valor de retorno opcional. El número de serie incluye el nombre (opcional) de un hilo. Todos los mensajes del mismo hilo se ordenan secuencialmente. Los mensajes de diversos hilos son concurrentes a menos que haya una dependencia secuencial explícita. Pueden añadirse varios detalles de implementación, como por ejemplo una distinción entre los mensajes síncronos y asíncronos.

Flujos. Generalmente, un diagrama de colaboración contiene un símbolo para un objeto durante una operación completa. Sin embargo, a veces, un objeto tiene distintos estados que se deban hacer explícitos. Por ejemplo, un objeto pudo cambiar de localización, o sus asociaciones pudieran diferenciarse perceptiblemente en distintos momentos. Un objeto se puede mostrar con su clase y su estado —un objeto de una clase en un estado—. El mismo objeto se puede mostrar múltiples veces, cada uno con una localización o estado diferentes.

Los diferentes símbolos de objeto que representan un objeto se pueden conectar usando flujos «**become**» o «**conversión**». Un flujo «**become**» es una transición, a partir de un estado de un objeto a otro. Se dibuja como una flecha de línea discontinua con el estereotipo «**become**» o «**conversión**» y puede ser etiquetado con un número de serie para mostrar cuando ocurre (Figura 8.4). Un flujo de conversión también se utiliza para mostrar la migración de un objeto a partir de una localización a otra distinta.

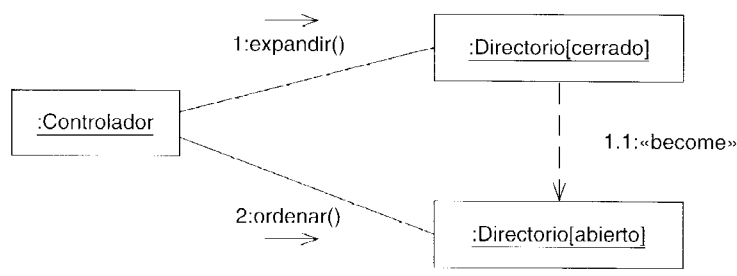


Figura 8.4 Flujos de conversión

Menos frecuentemente, el estereotipo «copy» o «copia» muestra un valor de objeto producido copiando otro valor de objeto.

La Tabla 8.1 muestra los tipos de relaciones de flujo de objeto.

Diagramas de colaboración y de secuencia. Los diagramas de colaboración y los diagramas de secuencia muestran interacciones, pero acentúan aspectos diferentes. Los diagramas de secuencia muestran secuencias de tiempo claramente pero no muestran explícitamente relaciones de objetos. Los diagramas de colaboración muestran las relaciones entre objetos claramente, pero las secuencias de tiempo se deben obtener a partir de números de secuencia. Los diagramas de secuencia son a menudo más útiles para mostrar escenarios; los diagramas de colaboración son a menudo más útiles para mostrar el diseño detallado de procedimientos.

Tabla 8.1 Tipos de Relaciones de Flujo

Flujo	Función	Notación
conversión	Transformación de un valor de un objeto a otro valor	«become» ----->
copia	Copia de un objeto que a partir de entonces es independiente	«copy» ----->

Patrones

Un patrón es una colaboración parametrizada, junto con las pautas sobre cuándo utilizarlo. Un parámetro se puede sustituir por diversos valores, para producir distintas colaboraciones. Los parámetros señalan generalmente las ranuras para las clases. Cuando se instancia un patrón, sus parámetros están ligados a clases reales de un diagrama de clase o a los roles dentro de una colaboración más amplia.

El uso de un patrón se representa como una elipse de línea discontinua conectada con cada una de sus clases por una línea discontinua, que se etiqueta con el nombre del rol. Por ejemplo, la Figura 8.5 muestra el uso del patrón **Observer** de [Gamma-95]. En este uso del patrón, **ColadeLlamadas** sustituye el rol del **sujeto** e **IconoDeBarraDeslizante** sustituye el rol del **manejador**.

Los patrones pueden aparecer en los niveles de análisis, arquitectura, diseño detallado e implementación. Son una manera de captar estructuras que ocurren frecuentemente para su reutilización. La Figura 8.5 muestra un uso del patrón Observer (Observador).

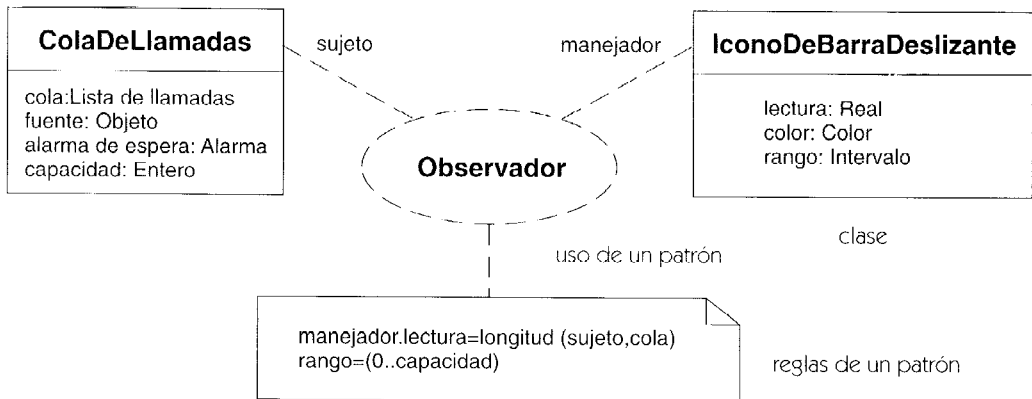


Figura 8.5 Uso del patrón

Descripción

Gran parte de un modelo de un sistema se piensa para mostrar los aspectos lógicos y de diseño del sistema, independiente de su empaquetado final en un medio de implementación. Sin embargo, los aspectos de implementación son importantes para los propósitos de reutilización y del rendimiento. UML incluye dos tipos de vistas para representar unidades de implementación: la vista de implementación y la vista de despliegue.

La vista de implementación muestra el empaquetado físico de las partes reutilizables del sistema en unidades sustituibles, llamadas componentes. Una vista de implementación muestra la implementación de los elementos del diseño (tales como clases) mediante componentes, así como sus interfaces y dependencias entre componentes. Los componentes son las piezas reutilizables de alto nivel a partir de las cuales se pueden construir los sistemas.

La vista de despliegue muestra la disposición física de los recursos de ejecución computacional, tales como computadores y sus interconexiones. Se llaman nodos. Durante la ejecución, los nodos pueden contener componentes y objetos. La asignación de componentes y de objetos a los nodos puede ser estática, o pueden migrar entre nodos.

La vista de despliegue puede mostrar cuellos de botella para el rendimiento si las instancias de los componentes con dependencias se ponen en distintos nodos.

Componente

Un componente es una unidad física de implementación con interfaces bien definidas pensada para ser utilizada como parte reemplazable de un sistema. Cada componente incorpora la implementación de ciertas clases del diseño del sistema. Los componentes bien diseñados no dependen directamente de otros componentes sino de las interfaces que ofrecen los componentes. En ese caso, un componente en un sistema se puede sustituir por otro componente que ofrezca las interfaces apropiadas.

Los componentes soportan más interfaces y requieren ciertas interfaces de otros componentes. Una interfaz es una lista de las operaciones que una pieza de software o de hardware ofrece y puede realizar. El uso de las llamadas interfaces permite evitar las dependencias directas entre componentes, facilitando una sustitución más fácil de nuevos componentes. La vista de componentes muestra la red de dependencias entre componentes. La vista de componentes puede aparecer de dos formas. Puede mostrar un conjunto de componentes disponibles (una biblioteca de componentes) con sus dependencias; éste es el material a partir del cual se puede ensam-

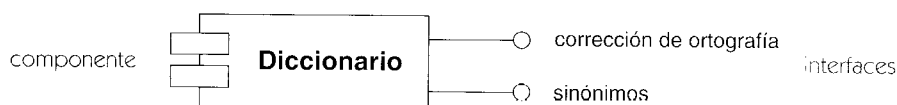


Figura 9.1 Componente con interfaces

blar un sistema. Puede también mostrar un sistema configurado, con la selección de componentes usados para construirlo (a partir de la biblioteca completa). De esta forma, cada componente se conecta a otros componentes cuyos servicios utiliza; estas conexiones deben ser consistentes con las interfaces de los componentes.

Un componente se dibuja como un rectángulo, con dos rectángulos pequeños a un lado. Puede ser unido por líneas sólidas a los círculos que representan sus interfaces (Figura 9.1).

Un diagrama de componentes muestra dependencias entre los componentes (Figura 9.2). Cada componente ofrece algunas interfaces y utiliza otras. Si las dependencias entre componentes se hacen a través de interfaces, los componentes se pueden sustituir por otros componentes que realicen las mismas interfaces.

Nodo

Un nodo es un objeto físico de ejecución que representa un recurso computacional, que generalmente tiene por lo menos memoria y a menudo también capacidad de proceso. Los nodos pueden tener estereotipos para distinguir diferentes tipos de recursos, tales como UCP, dispositivos, y memorias. Los nodos pueden contener objetos, instancias, instancias del componente.

Un nodo se representa mediante un cubo estilizado con el nombre del nodo y, opcionalmente, su clasificación (Figura 9.3).

Las asociaciones entre los nodos representan líneas de comunicación. Las asociaciones pueden tener estereotipos para distinguir diversos tipos de enlaces.

Los nodos pueden tener relaciones de generalización para relacionar una descripción general de un nodo con una variación más específica.

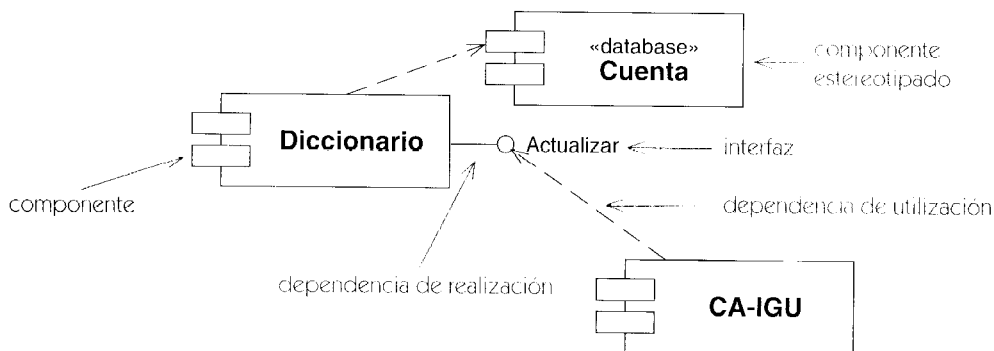


Figura 9.2 Diagrama de componentes

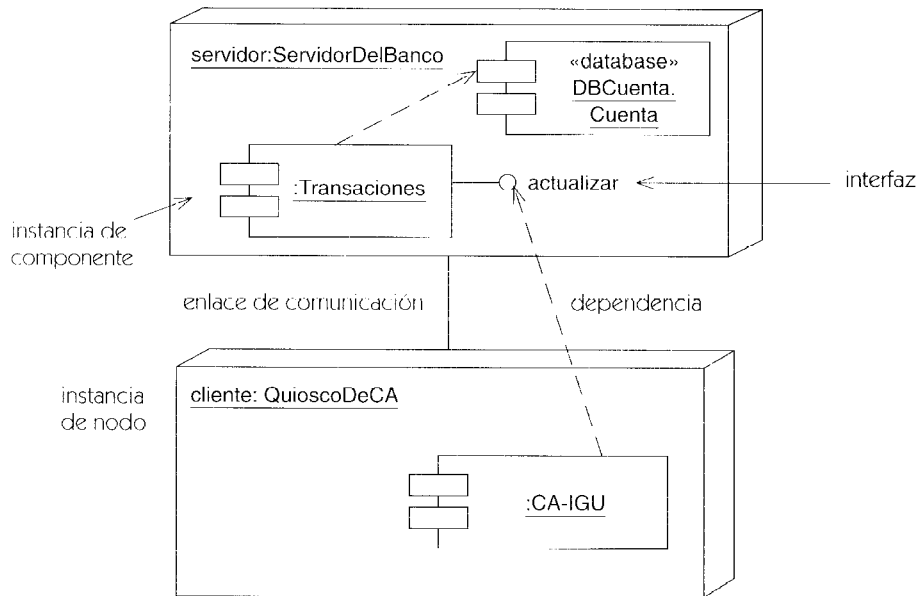


Figura 9.3 Diagrama de despliegue

La presencia de un objeto en un nodo se representa mediante el anidamiento físico del símbolo de objeto dentro del símbolo del nodo. Si eso no es conveniente, el símbolo de objeto puede contener la etiqueta **location**, cuyo valor sea el nombre del nodo en el que reside el objeto (su localización). La migración de objetos o de instancias de componentes entre nodos, puede representarse también.

Véase conversión.

Descripción

Cualquier sistema grande se debe dividir en unidades más pequeñas, de modo que las personas puedan trabajar con una cantidad de información limitada, a la vez y de modo que los equipos de trabajo no interfieran con el trabajo de los otros. La gestión del modelo consiste en paquetes (incluyendo tipos especiales de paquetes) y relaciones de dependencia entre paquetes.

Paquete

Un paquete es una parte de un modelo. Cada parte de un modelo debe pertenecer a un paquete. El modelador puede asignar el contenido de un modelo a un conjunto de paquetes. Pero, para ser funcional, la asignación debe seguir un cierto principio racional, tal como funcionalidad común, implementación estrechamente relacionada, y un punto de vista común. UML no impone una regla para componer los paquetes, pero una buena descomposición en paquetes realzará enormemente la capacidad de mantenimiento del modelo.

Los paquetes contienen elementos del modelo al más alto nivel, tales como clases y sus relaciones, máquinas de estado, diagramas de casos de uso, interacciones, y colaboraciones: cualquier elemento que no esté contenido en otro. Los elementos como atributos, operaciones, estados, líneas de vida, y mensajes están contenidos en otros elementos, y no aparecen como contenido directo de los paquetes. Cada elemento de nivel superior tiene un paquete en el cual se declara. Éste es su paquete de origen. Puede hacerse referencia a él en otros paquetes, pero el contenido del elemento pertenece al paquete de origen. En un sistema de control de configuración, un modelador debe tener acceso al paquete de origen para modificar el contenido de un elemento. Esto proporciona un mecanismo de control de acceso para trabajar en los modelos grandes. Los paquetes son también las unidades para cualquier mecanismo de versiones.

Los paquetes pueden contener otros paquetes. Hay un paquete raíz, que contiene indirectamente el modelo completo de un sistema. Hay varias maneras posibles de organizar los paquetes en un sistema. Pueden ser organizados por la vista, por la funcionalidad, o por cualquier otra base que el modelador elija. Los paquetes son unidades de organización jerárquica de uso general de los modelos de UML. Pueden ser utilizados para el almacenamiento, el control de acceso, la gestión de la configuración, y la construcción de bibliotecas que contengan fragmentos reutilizables del modelo.

Si se eligen bien los paquetes, reflejan la arquitectura de alto nivel de un sistema: su descomposición en subsistemas y sus dependencias. Una dependencia entre paquetes resume las dependencias entre los contenidos del paquete.

Dependencias en los paquetes

Las dependencias que se presentan entre elementos individuales, pero en un sistema de cualquier tamaño, deben ser vistas en un nivel más alto. Las dependencias entre los paquetes resumen dependencias entre los elementos internos a ellos, es decir, las dependencias del paquete son derivables a partir de las dependencias entre elementos individuales.

La presencia de una dependencia entre paquetes implica que existe en un enfoque ascendente (una declaración de existencia), o que se permite que exista más adelante en un enfoque descendente (una restricción que limita cualquier otra relación), por lo menos un elemento de relación con el tipo indicado de dependencia entre elementos individuales dentro de los paquetes correspondientes. Es una declaración de existencia y no implica que todos los elementos del paquete tengan la dependencia. Es un indicador para el modelador de que existe información adicional, pero la dependencia, cuando se trata de paquetes, no contiene más información por sí misma; es solamente un resumen.

El enfoque descendente refleja la arquitectura del sistema total. El enfoque ascendente se puede generar automáticamente a partir de los elementos individuales. Ambos enfoques tienen su lugar en el modelado, incluso en un solo sistema.

Las dependencias múltiples del mismo tipo entre elementos individuales se agregan a una sola dependencia entre los paquetes que contienen los elementos. Si las dependencias entre elementos individuales contienen estereotipos (tales como diferentes tipos de uso), el estereotipo se puede omitir en la dependencia de paquete, para dar una sola dependencia de alto nivel.

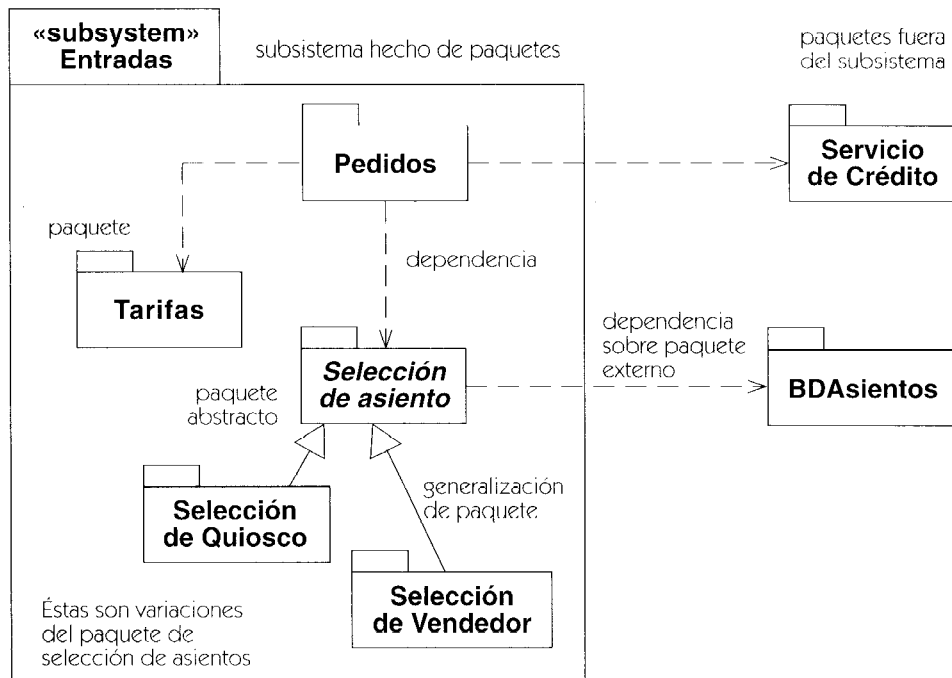


Figura 10.1 Paquetes y sus relaciones

Los paquetes se dibujan como rectángulos con lengüetas (iconos “carpeta”). Las dependencias se muestran como flechas con líneas discontinuas.

La Figura 10.1 muestra la estructura del paquete para un subsistema de reserva de entradas. Tiene dependencias en los paquetes exteriores y dos variaciones del paquete de **selección de asientos**. Cualquier implementación del subsistema incluiría solamente una variación.

Dependencia de acceso e importación

En general, un paquete no puede tener acceso al contenido de otro paquete. Los paquetes son opacos, a menos que sean abiertos por una dependencia de acceso o de importación. La dependencia de acceso se aplica directamente a los paquetes y a otros contenedores. En el nivel de paquete, la dependencia de acceso indica que el contenido del paquete del proveedor puede aparecer en referencias efectuadas por los elementos del paquete cliente o de los paquetes incluidos dentro del cliente. Un elemento en el proveedor debe tener suficiente visibilidad dentro de su paquete para permitir a un cliente verlo. En general, un paquete puede ver solamente los elementos de otros paquetes que tienen asignada visibilidad pública en el paquete que los contiene. Los elementos con visibilidad protegida son visibles solamente por los paquetes que son descendientes del paquete que contiene los elementos. Los elementos con visibilidad privada son visibles solamente por el paquete que los contiene, y por cualesquiera paquetes anidados en el interior de ese paquete. La visibilidad también se aplica al contenido de clases (atributos y operaciones). Un descendiente de una clase puede ver a miembros de su antecesor con visibilidad pública o protegida; cualquier otra clase puede ver solamente miembros con visibilidad pública. El permiso de acceso y la visibilidad apropiada son necesarios para hacer referencia a un elemento. Así que para que un elemento, en un paquete, vea a un elemento en un paquete sin relación, el primer paquete debe tener acceso o importar al segundo paquete, y el elemento destino debe tener visibilidad pública dentro del segundo paquete.

Un paquete anidado dentro de otro paquete es parte del contenedor y tiene acceso completo a su contenido, sin necesidad de accesos. El contenedor, sin embargo, puede no ver el interior de sus paquetes anidados si no tiene acceso. Se encapsula el contenido.

Observe que una dependencia de acceso no modifica el espacio de nombres del cliente ni crea las referencias automáticamente de ninguna otra forma. Simplemente concede permiso para establecer referencias. La dependencia de importación se utiliza para agregar nombres al espacio de nombres del paquete del cliente como sinónimos de los caminos completos.

Modelo y subsistema

Un modelo es un paquete que abarca una descripción completa de una vista particular de un sistema. Proporciona una descripción cerrada de un sistema a partir de un punto de vista. No tiene dependencias fuertes en otros paquetes, tales como dependencias de implementación o dependencias de herencia. La relación de traza es una forma débil de dependencia entre elementos, en distintos modelos, que observan la presencia de una cierta conexión sin implicaciones semánticas específicas.

Generalmente, un modelo se estructura con forma de árbol. El paquete raíz contiene anidados en sí mismo paquetes que constituyen el detalle completo del sistema desde un punto de vista dado.

Un subsistema es un paquete que tiene piezas separadas de especificación y de realización. Representa una unidad coherente del modelo, con interfaces limpias al resto del sistema. Representa generalmente la partición del sistema, en un límite funcional o de implementación. Los modelos y los subsistemas se dibujan como paquetes con las palabras clave de estereotipo (Figura 10.1).

Descripción

UML proporciona varios mecanismos de extensión para permitir que los modeladores hagan algunas extensiones comunes sin tener que modificar el lenguaje de modelado subyacente. Se han diseñado estos mecanismos de extensión de modo que las herramientas puedan almacenar y manipular las extensiones aun sin entender su semántica completa. Por esta razón, las extensiones se pueden almacenar y manipular como cadenas. Para una herramienta que no entienda la extensión, es sólo una cadena, pero se puede introducir o almacenar como parte de un modelo, y ser pasada a otras herramientas. Se espera que las herramientas de base y los módulos adicionales sean escritos para procesar varios tipos de extensiones. Estas herramientas definirán una sintaxis y una semántica particulares para sus extensiones que sólo ellas necesitan entender.

Este acercamiento a las extensiones no resolverá probablemente todas las necesidades que se presenten, pero pensamos que se ajustará a una porción grande de las adaptaciones que necesitan la mayoría de los modeladores de una manera simple que es fácil de implementar. Los mecanismos de extensión son restricciones, valores etiquetados, y estereotipos. Tenga presente que una extensión, por definición, desvía la forma de estándar de UML y puede, por lo tanto, conducir a problemas de interoperatividad. El modelador debe sopesar cuidadosamente ventajas y costes antes de usar extensiones, especialmente cuando los mecanismos existentes trabajaran razonablemente bien. Típicamente, las extensiones están pensadas para los dominios de aplicaciones particulares o los entornos de programación, pero dan lugar a un dialecto de UML, con las ventajas y desventajas de todos los dialectos.

Restricción

Una restricción es una condición semántica representada como expresión textual. Cada expresión tiene un lenguaje implícito de interpretación, que puede ser una notación matemática formal, tal como notación de teoría de conjuntos; un lenguaje computarizado de restricción, tal como OCL; un lenguaje de programación, como C++; o pseudocódigo o el lenguaje natural informal. Por supuesto, si el lenguaje es informal, entonces su interpretación es también informal, y debe ser hecha por una persona. Incluso si una restricción se expresa en un lenguaje formal, no significa que se hará cumplir automáticamente. Las técnicas de mantenimiento y comprobación automática de verdades no pueden cubrir la mayoría de los casos, pero por lo menos la semántica será exacta.

Las restricciones pueden expresar limitaciones y relaciones que no se pueden expresar usando la notación de UML. Son particularmente útiles para indicar las condiciones globales o las condiciones que afectan a ciertos elementos.

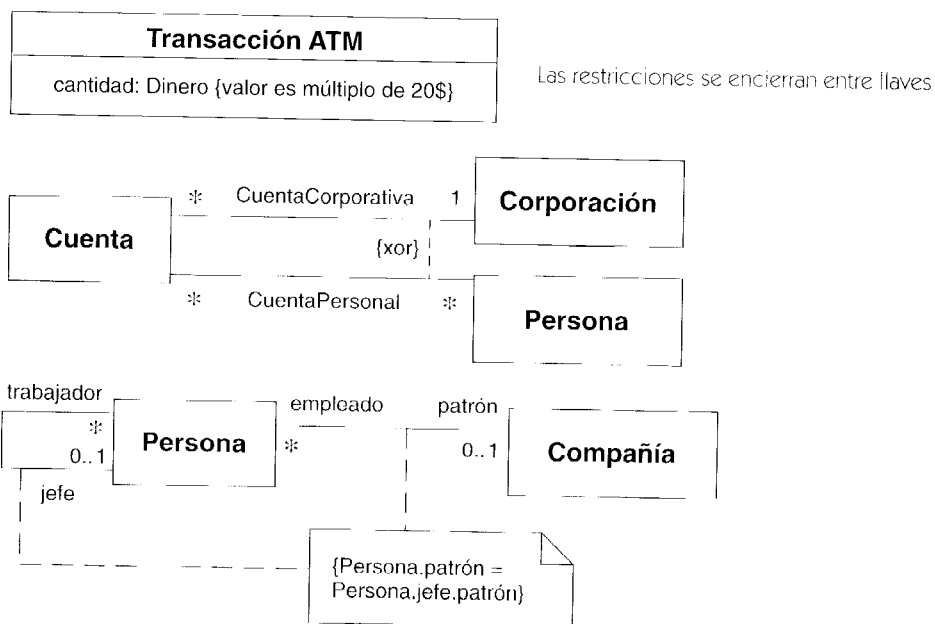


Figura 11.1 Restricciones

Las restricciones se muestran como expresiones de cadena entre llaves. Pueden ser añadidas a un elemento de una lista, ser asociadas a una dependencia, o ser incluidas en una nota. La Figura 11.1 muestra varios tipos de restricciones.

Valor etiquetado

Un valor etiquetado es un par de cadenas: una cadena de etiqueta y una cadena de valor, que almacena un elemento de información sobre un elemento. Un valor etiquetado se puede unir a cualquier elemento individual, incluyendo los elementos del modelo y los elementos de la presentación.

La etiqueta es un nombre de una cierta propiedad que el modelador desea registrar, y el valor es el valor de esa propiedad para el elemento dado. Por ejemplo, la etiqueta pudo ser **autor**, y el valor pudo ser el nombre de la persona responsable del elemento, tal como **Charles Babbage**.

Los valores etiquetados se pueden utilizar para almacenar cualquier información sobre los elementos. Son particularmente útiles para almacenar información de gestión del proyecto, tal como la fecha de creación de un elemento, de su estado de desarrollo, de fechas debidas, y del estado de la prueba. Cualquier cadena se puede utilizar como un nombre de etiqueta, excepto aquellos nombres de atributos incorporados al metamodelo que deben evitarse (porque las etiquetas y los atributos juntos pueden considerarse propiedades de un elemento y accederse de manera uniforme en una herramienta), y ciertos nombres de etiqueta están predefinidos (véase el capítulo 14, Elementos Estándar).

Los valores etiquetados también proporcionan una manera de añadir información dependiente de la implementación a los elementos. Por ejemplo, un generador de código necesita la información adicional sobre el tipo de código a generar a partir de un modelo. A menudo, hay

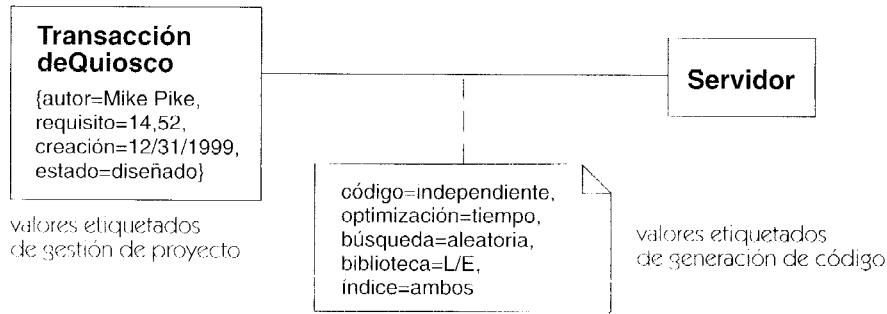


Figura 11.2 Valores etiquetados

varias maneras posibles de implementar correctamente un modelo; el modelador debe proporcionar las directrices sobre qué elección tomar. Ciertas etiquetas se pueden utilizar como indicadores para decir al generador de código qué implementación utilizar. Otras etiquetas se pueden utilizar para otros tipos de herramientas añadidas, tales como planificadores del proyecto y generadores de informes.

Los valores etiquetados también se pueden utilizar para almacenar información sobre elementos estereotipados del modelo (discutidos abajo).

Los valores etiquetados se muestran como cadenas con el nombre de la etiqueta, un igual, y el valor. Se colocan normalmente en listas entre llaves (Figura 11.2). Serán omitidos en los diagramas pero mostrados a menudo en listas y formularios emergentes.

Estereotipos

Muchos modeladores desean adaptar un lenguaje de modelado para un dominio de aplicación particular. Esto conlleva un cierto riesgo, porque el lenguaje adaptado no será universalmente comprensible, pero existe, sin embargo, la tentación de hacerlo.

Un estereotipo es un tipo de elemento del modelo definido en el propio modelo. El contenido de información y la forma de un estereotipo son igual que los de una clase existente de elemento del modelo base, pero su significado y uso son diferentes. Por ejemplo, los modeladores en el área de modelado de negocio a menudo quieren distinguir objetos de negocio y procesos del negocio como tipos especiales de elementos de modelado cuyo uso es distinto dentro de un proceso dado del desarrollo. Éstos se pueden tratar como tipos especiales de clases: tienen atributos y operaciones, pero tienen restricciones especiales en sus relaciones con otros elementos y en su uso.

Un estereotipo se basa en un elemento de modelado existente. El contenido de información del elemento estereotipado es igual que el elemento de modelo existente. Esto permite que una herramienta almacene y manipule el nuevo elemento de la misma forma que lo hace con el elemento existente. El elemento estereotipado puede tener sus propios iconos; es fácil que una herramienta lo permita. Por ejemplo, una "organización de negocios" podría tener un icono que parece un grupo de personas. El estereotipo puede también tener una lista de las restricciones que se aplican a su uso. Por ejemplo, quizás una "organización de negocios" se puede asociar solamente a otra "organización de negocios" y no a cualquier clase. No todas las restricciones se pueden verificar automáticamente por una herramienta de propósito general, pero

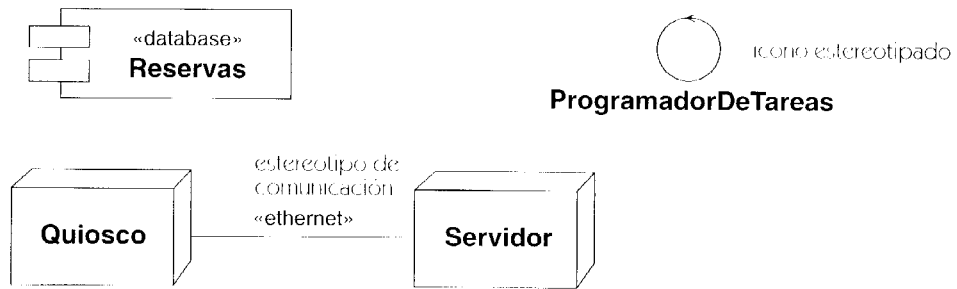


Figura 11.3 Estereotipos

pueden hacerse cumplir manualmente o ser verificadas por una herramienta añadida que entienda el estereotipo.

Los estereotipos pueden utilizar valores etiquetados para almacenar las propiedades adicionales que no se pueden modelar con el elemento base.

Los estereotipos se muestran como cadenas de texto rodeadas por comillas («») puestos dentro o cerca del símbolo normal del elemento. El modelador puede también crear un icono para un estereotipo particular, que sustituye el símbolo normal del elemento (Figura 11.3).

Adaptación de UML

Los mecanismos de extensión de restricciones, de valores etiquetados, y de estereotipos permiten adaptar los perfiles de UML para los dominios especiales de uso. Se han realizado varios perfiles, que se describen en el apéndice C, Extensiones de Proceso. Otros han sido propuestos por los usuarios. La capacidad de adaptación del lenguaje de modelado significa que los dominios de uso pueden adaptar el lenguaje de modelado a sus necesidades, sin embargo, todavía comparte la extensa preponderancia de los conceptos genéricos y comunes a todos los dominios.

Descripción

Los modelos de UML se utilizan dentro de un entorno. La mayoría de la gente utiliza el modelado como medio para un fin —vamos a llamarlo desarrollo de buenos sistemas— y no como fin en sí mismo. El propósito y la interpretación del modelo están afectados por el resto del entorno. Otras facilidades en el entorno más amplio incluyen los metamodelos que abarcan muchos lenguajes, herramientas de edición del modelo, lenguajes de programación, sistemas operativos y componentes importantes del sistema, y el mundo de los negocios y la ingeniería dentro del cual se utilizan los sistemas. La responsabilidad de dar significado a un modelo y de implementar su objetivo se resuelve con todas estas facilidades, incluyendo UML.

Los modelos existen en varios niveles de concreción. UML es un lenguaje de modelado, de propósito general, que incluye semántica y notación, pero es utilizable con diversas herramientas y lenguajes de implementación. Cada nivel de uso introduce ciertas consideraciones de modelado, que aparecen en UML en diferentes grados.

Responsabilidades semánticas

Un metamodelo es la descripción de un modelo. Un lenguaje de modelado describe modelos; por lo tanto, puede ser descrito por un metamodelo. Un metamodelo procura hacer un lenguaje exacto, definiendo su semántica, pero hay una tensión para permitir las extensiones para nuevas situaciones. La forma real del metamodelo es importante para la implementación de las herramientas, y el intercambio de modelos, pero no muy importante para la mayoría de los usuarios. Por lo tanto no la hemos incluido en este libro. Los que estén interesados pueden consultar los documentos originales del estándar [UML-9] disponibles en el CD adjunto.

Un metamodelo y un lenguaje deben cubrir muchos campos y acomodar muchas interpretaciones. Los sistemas existentes tienen modelos de ejecución y de memoria diferentes. Es imposible elegir uno de ellos como la interpretación correcta. De hecho, es probablemente engañoso incluso considerar tal opción. En su lugar, uno puede pensar en las diversas interpretaciones de los modelos de ejecución como puntos de variación semántica. Un punto de variación semántica es un punto de diferencia sobre la semántica detallada de la ejecución, pero que es ortogonal a otros aspectos de un sistema. Por ejemplo, un entorno puede elegir o no ofrecer la clasificación dinámica, la capacidad de un objeto de cambiar su clase en tiempo de ejecución. Hoy, la mayoría de los lenguajes de programación no lo permiten, principalmente por razones de implementación del lenguaje de programación, pero algunos sí. La diferencia es indistinguible en la semántica estática. La opción de la clasificación estática o de la clasificación dinámica se puede identificar

como un punto de variación semántica con dos opciones: clasificación estática o clasificación dinámica. Cuando existen tales opciones, la gente discute a menudo sobre cuál es la interpretación correcta. En vez de esto, observe que esto es una elección y asígnele un nombre para poder utilizarla.

Un metamodelo describe el contenido de un modelo bien formado, mientras que un lenguaje de programación describe un programa bien formado. Solamente un modelo bien formado tiene un significado y una semántica apropiada; no tiene sentido pedir el significado de un modelo mal formado. Sin embargo, la mayoría del tiempo, los modelos bajo desarrollo no están bien formados. Son incompletos y posiblemente inconsistentes. Pero esto es lo que deben contemplar las herramientas de modelado: modelos incompletos, no sólo modelos acabados. El metamodelo de UML describe modelos correctos, bien formados. Un metamodelo separado puede describir posibles fragmentos de un modelo. Dejamos a los fabricantes de la herramienta decidir dónde situar el límite en los fragmentos del modelo admitidos, y qué tipo de semántica dar a los modelos mal formados.

UML incluye algunos mecanismos de extensión incorporados para adaptar su uso en dominios especializados. Los mecanismos incluyen la capacidad de definir estereotipos y valores etiquetados. Estos mecanismos se pueden utilizar para adaptar una variante de UML, definiendo un sistema de estereotipos y de etiquetas, y adoptando las convenciones para su uso a la hora de construir un modelo. Por ejemplo, las variantes desarrolladas podrían centrarse en la semántica de la implementación de varios lenguajes de programación. La adición de extensiones puede ser poderosa, pero conlleva algunos peligros inherentes. Dado que su semántica no se define dentro de UML, UML no puede proveer su significado; la interpretación se delega al modelador. Además, si no se tiene cuidado, algunos significados pueden ser ambiguos o incluso inconsistentes. Las herramientas pueden proporcionar significados y funciones automatizadas para los estereotipos y las etiquetas definidas por las herramientas, pero no para las extensiones definidas por el usuario. Sin importar la ayuda para las extensiones, cualquier extensión tira del usuario lejos del centro común que el estándar del lenguaje proporciona y socava las metas de la capacidad de intercambio de modelos y de comprensibilidad de los modelos. Por supuesto, siempre que usted utilice una biblioteca de clases particular, usted diverge de la capacidad de intercambio perfecta de la nada. Así que, no se preocupe sobre lo abstracto. Utilice las extensiones cuando sean de ayuda, pero evítelas cuando no sean necesarias.

Responsabilidades de notación

La notación no agrega significado a un modelo, sino que ayuda al usuario a entender su significado. La notación no tiene semántica, sino que agrega a menudo las connotaciones para un usuario, tal como la afinidad percibida de dos conceptos basados en su proximidad en un diagrama.

Los documentos de UML [UML-98] y este libro definen una notación canónica de UML, que podría llamarse formato de publicación para modelos. Esto es similar a muchos lenguajes de programación en los cuales los programas dentro de artículos publicados se imprimen en un formato atractivo con disposición cuidadosa, palabras reservadas en negrita, y separan las figuras para cada procedimiento. Los verdaderos compiladores tienen que aceptar una entrada más sucia. Esperamos que esas herramientas de edición extiendan la notación a un formato de pantalla, incluyendo las cosas tales como el uso de tipografías y de color para resaltar elementos; la capacidad de suprimir y de filtrar fácilmente elementos que no son actualmente de interés, para

ampliar un diagrama, para mostrar los elementos anidados, para atravesar enlaces a otros modelos o vistas; y animación. Sería imposible intentar estandarizar todas estas posibilidades y absurdo intentarlo, porque no hay necesidad y limitaría la innovación útil. Esta clase de extensiones de notación es responsabilidad del constructor de la herramienta. En una herramienta interactiva hay menos peligro de ambigüedad, porque el usuario puede pedir siempre una aclaración. Esto es probablemente más útil que insistir en una notación que sea totalmente inequívoca en el primer vistazo. Lo importante es que una herramienta debe poder producir la notación canónica cuando ésta sea solicitada, especialmente en forma impresa, y que una herramienta interactiva debe suministrar extensiones razonables a este comportamiento mínimo.

Contamos con que las herramientas también permitan que los usuarios extiendan la notación de maneras limitadas pero útiles. Hemos especificado que los estereotipos pueden tener sus propios iconos. Podrían estar permitidas otras clases de extensiones de notación, pero los usuarios deben aplicar un buen criterio.

Observe que la notación es algo más que figuras; incluye la información en forma de texto y los hiperenlaces invisibles entre elementos de representación.

Responsabilidades del lenguaje de programación

UML debe trabajar con varios lenguajes de implementación, sin incorporarlos explícitamente. Pensamos que UML debe permitir el uso de cualquier (o por lo menos de muchos) lenguajes de programación, para la especificación y la generación del código final. El problema es que cada lenguaje de programación tiene muchos elementos semánticos, que no deseamos absorber en UML, porque se manejan mejor como elementos del lenguaje de programación, y hay variaciones considerables en la semántica de ejecución. Por ejemplo, la semántica de concurrencia se maneja de maneras diferentes entre los lenguajes (si es que se maneja en todos).

Los tipos de datos primitivos no se describen detalladamente en UML. Esto es deliberado pues no deseábamos incorporar la semántica de un lenguaje de programación en preferencia a todos los demás. Para la mayoría de los propósitos de modelado, esto no es un problema. Utilice el modelo semántico aplicable a su lenguaje. Éste es un ejemplo de un punto de variación semántico.

La representación de las propiedades detalladas del lenguaje para la implementación suscita el problema de capturar la información sobre propiedades de la implementación sin la construcción de su semántica en UML. La decisión tomada fue capturar las propiedades del lenguaje que van más allá de capacidades incorporadas en UML, por medio de estereotipos y de valores etiquetados. Éstos se pueden asignar a las propiedades del lenguaje y a las opciones de la generación de código por una herramienta o un generador de código. Un editor genérico no necesita entenderlos. Un usuario podría, de hecho, crear un modelo usando una herramienta que no contemple el lenguaje a utilizar, y transferir el modelo final a otra herramienta para el procesamiento final. Por supuesto, si la herramienta no entiende los estereotipos y las etiquetas, no puede comprobarlas para saber si hay consistencia. Pero esto no es peor que la práctica normal con los editores de textos y los compiladores. En caso de necesidad, se puede crear una herramienta que utilice un conjunto particular de extensiones.

La generación de código y la ingeniería inversa requerirán, al menos en el futuro próximo, la participación del diseñador además de un modelo de UML. Las directivas y ayudas al generador de código se pueden proveer como valores etiquetados y estereotipos. Por ejemplo, el modelador

podría indicar qué tipo de clase contenedora se debe utilizar para implementar una asociación. Por supuesto, esto significa que los ajustes para generación de código en las herramientas pueden ser incompatibles, pero no creemos que haya actualmente suficiente acuerdo sobre la forma correcta de estandarizar los ajustes reales. En cualquier caso, diversas herramientas utilizarán sus generadores de código como su ventaja competitiva. Eventualmente, los ajustes por defecto pueden emerger y llegar a estar maduros para la estandarización.

Modelado con herramientas

Para modelar sistemas de tamaño real se necesitan herramientas. Las herramientas proporcionan maneras interactivas de ver y editar modelos. Proporcionan un nivel de organización, que está fuera del alcance del UML por sí mismo, pero que permite entender al usuario y ayuda en el acceso a la información. Las herramientas ayudan a encontrar la información en modelos grandes, buscando y filtrando lo que se presenta.

Algunos aspectos sobre el uso de herramientas

Las herramientas se ocupan de la organización y del almacenamiento físico de los modelos. Deben dar soporte múltiples equipos de trabajo en un sólo proyecto, así como permitir la reutilización entre proyectos. Los siguientes elementos están fuera del alcance del UML canónico, pero se deben considerar para el uso real de la herramienta.

Ambigüedades e información sin especificar. En las etapas iniciales faltan aún muchas cosas por decir. Las herramientas deben poder ajustar la precisión de un modelo y no forzar cada valor para ser específico. Véanse las siguientes secciones “Modelos inconsistentes para el trabajo en marcha” y “Valores sin especificar y nulos.”

Opciones de presentación. Los usuarios no desean ver toda la información todo el tiempo. Las herramientas deben permitir filtrar y ocultar la información no deseada en un momento dado, así como visualizaciones alternativas, usando las capacidades de visualización del hardware disponible. Esto se ha descrito ya, en la sección “Responsabilidades de la Notación.”

Gestión del modelo. El control de configuración, el control de acceso, y la gestión de versiones de las unidades del modelo está fuera del alcance de UML, pero son cruciales para el proceso de ingeniería del software y están en la cima del metamodelo.

Interfaces con otras herramientas. Los modelos necesitan ser manejados por los generadores de código, las calculadoras de métricas, los escritores de informes, los motores de ejecución, y otras herramientas de base. La información para otras herramientas necesita ser incluida en los modelos, pero no es información de UML. Los valores etiquetados son apropiados para contener esta información.

Modelos inconsistentes para el trabajo en marcha

La última meta del modelado es producir una descripción de un sistema en un cierto nivel de detalle. El modelo final debe satisfacer varias restricciones de validación para ser significativo. Sin embargo, como en todo proceso creativo, el resultado no se produce necesariamente de forma lineal. Los productos intermedios no satisfarán todos las restricciones de validez en cada paso. En la práctica, una herramienta debe manejar no sólo semánticamente los modelos válidos

que satisfacen las restricciones de validez, sino también los modelos sintácticamente válidos, que satisfacen ciertas reglas de construcción pero pueden violar algunas restricciones de validez. Los modelos semánticamente inválidos no son directamente utilizables. En su lugar pueden considerarse como “trabajos en marcha”, que representan las trayectorias hacia el resultado final.

Valores sin especificar y nulos

Un modelo completo debe tener valores para todos los atributos de sus elementos. En muchos casos, *null o nulo* (ningún valor) es uno de los valores posibles, pero si un valor puede ser nulo o no, debe ser parte de la descripción del tipo del atributo; muchos tipos no tienen un valor nulo natural dentro de su gama de valores. Por ejemplo, nulo no tiene ningún sentido como límite superior en el tamaño de un conjunto. O el conjunto tiene un tamaño superior fijo o no hay límite, en cuyo caso su tamaño máximo es ilimitado, así que la posibilidad de ser nulo es un añadido al rango de posibles valores de un tipo de datos.

Por otra parte, durante las primeras etapas del diseño, un desarrollador puede no cuidar el valor de una propiedad en concreto. Puede ser que sea un valor que no es significativo en una etapa particular, por ejemplo, la visibilidad al hacer un modelo del dominio. O el valor puede ser significativo pero el modelador pudo no haberlo especificado todavía, y el desarrollador necesita recordar que aún es necesario elegir su valor. En este caso, el valor está sin especificar. Esto indica que un valor será eventualmente necesario, pero que todavía no se ha especificado. No es igual que un valor nulo, que puede ser un valor legítimo en el modelo final. En muchos casos, particularmente con las cadenas, un valor nulo es una buena manera de indicar un valor sin especificar, pero no son lo mismo. Un valor sin especificar no es significativo en un modelo bien formado. La definición de UML no maneja valores sin especificar. Son responsabilidad de las herramientas que soportan UML, y se consideran parte de un trabajo en marcha que, por necesidad, no tiene ningún significado semántico.

Parte 3: Referencia



abstracción

1. Acto de identificar aquellas características esenciales de una cosa que la distinguen del resto de cosas. La abstracción implica buscar parecidos en conjuntos de cosas, centrándose en las características esenciales comunes. Una abstracción siempre implica la perspectiva de quien la realiza, por lo que pueden existir diferentes abstracciones para la misma cosa según el propósito. Todo modelado implica abstracción, a menudo en diferentes niveles para propósitos distintos.
2. Tipo de dependencia que relaciona dos elementos que representan el mismo concepto en diferentes niveles de abstracción.

Véase derivación, realización, refinamiento, traza.

Semántica

Una dependencia de abstracción es una relación entre dos elementos que se encuentran en niveles distintos de abstracción, como las representaciones en diferentes modelos, en niveles distintos de precisión, en niveles distintos de concreción o en niveles distintos de optimización. Generalmente las dos representaciones no se utilizarán simultáneamente. Normalmente un elemento es más detallado que otro, donde el más detallado es el cliente y el otro el proveedor. Si no está claro qué elemento es más detallado, se puede modelar cualquiera de los dos elementos como cliente.

Los estereotipos de dependencia de abstracción son la traza (palabra clave «**trace**»), el refinamiento (palabra clave «**refine**»), la realización (palabra clave «**realize**») y la derivación (palabra clave «**derive**»).

Notación

Una dependencia de abstracción se representa mediante una flecha con línea discontinua que va del elemento cliente al elemento proveedor etiquetada con alguna de las palabras clave «**trace**», «**refine**» o «**derive**». La dependencia de realización tiene su propia notación, que consiste en una flecha con línea discontinua cuyo extremo es un triángulo cerrado que apunta al elemento proveedor.

La correspondencia entre elementos se puede adjuntar a la relación como restricción.

Elementos estándar

derive, refine, trace

abstracto/a

Clase, caso de uso, señal, otro clasificador u otro elemento generalizable que no puede ser instanciado directamente. También se utiliza para describir una operación que no tiene implementación. Antónimo: concreto/a.

Véase operación abstracta, elemento generalizable.

Semántica

Una clase abstracta es una clase no instanciable, es decir, que no puede tener instancias directas, bien porque su descripción es incompleta (le falta el método de una o más operaciones) o porque no ha sido pensada para ser instanciada aunque su descripción esté completa. El objetivo fundamental de las clases abstractas es la especialización. Para ser útil, una clase abstracta debe tener descendientes que puedan tener instancias: una clase hoja abstracta no sirve para nada. (Pueden aparecer clases abstractas como hojas en un marco de trabajo, pero al final deben especializarse.)

Una clase concreta no debe tener operaciones abstractas, pues de lo contrario sería necesariamente abstracta, pero una clase abstracta sí puede tener operaciones concretas. Las operaciones concretas son aquellas que pueden implementarse una vez y utilizarse tal cual en toda la jerarquía de subclases. En su implementación, las operaciones concretas sólo pueden utilizar características (atributos y operaciones) conocidas por la clase en la que se declaran. Uno de los propósitos de la herencia es situar estas operaciones en superclases abstractas para que puedan ser compartidas por todas las subclases. Una operación concreta puede ser polimórfica, es decir, puede ser sobrescrita por un método de una clase descendiente, pero no es necesario que lo sea (puede ser una operación hoja). Una clase cuyas operaciones están todas implementadas puede ser abstracta, pero debe ser declarada explícitamente como tal. Una clase con una o más operaciones no implementadas es automáticamente abstracta.

La misma semántica se aplica a los casos de uso. Un caso de uso abstracto define un fragmento de comportamiento que no puede aparecer por sí mismo, pero que puede aparecer en la definición de casos de uso concretos a través de relaciones de generalización, extensión o inclusión. Llevando el comportamiento común a un caso de uso abstracto, el modelo se reduce y es más fácil de comprender.

Existe una relación similar en otros clasificadores y elementos generalizables.

Notación

El nombre de una clase u operación abstracta debe aparecer en cursiva. También se puede utilizar la palabra **abstract** en la lista de propiedades que aparece después o debajo del nombre, por ejemplo, **Cuenta {abstract}**.

Véase también nombre de clase.

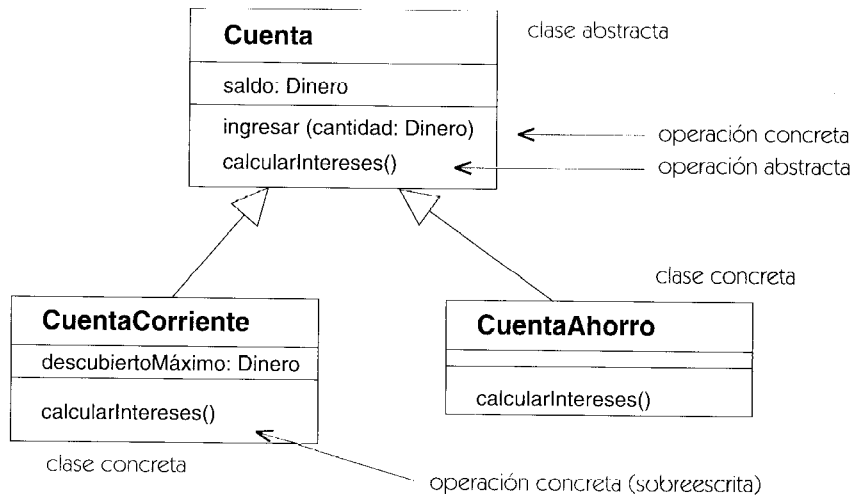


Figura 13.1 Clases abstractas y concretas

Ejemplo

La Figura 13.1 muestra una clase abstracta **Cuenta** con una operación abstracta —**calcularIntereses**— y una operación concreta —**ingresar**—, de la que se han declarado dos subclases concretas. Debido a que las subclases son concretas, cada una de ellas puede implementar la operación **calcularIntereses**. Los atributos son siempre concretos.

Discusión

La distinción entre modelar una clase como abstracta o como concreta no es tan importante como a primera vista podría parecer, pues es más una decisión de diseño sobre un modelo que una propiedad inherente al mismo. El estado de una clase puede cambiar con la evolución de un diseño. Una clase concreta puede modelarse como abstracta si se añaden subclases que incluyan todas sus posibilidades. Una clase abstracta puede ser modelada como concreta si nos encontramos con que las diferencias entre sus subclases son innecesarias y por tanto deben ser eliminadas, o si dichas diferencias pueden representarse utilizando diferentes valores de un atributo en lugar de subclases distintas.

Una forma de simplificar la decisión es adoptar el principio de diseño que afirma que todas las clases que no son hojas deben ser abstractas (y todas las clases hoja deben ser necesariamente concretas, excepto aquellas clases hoja pensadas para generalizaciones futuras). Esta regla no pertenece a UML: es un estilo que se puede o no adoptar. La razón de ser de esta “regla de la superclase abstracta” son las diferencias entre un método heredable de una superclase y un método de una clase concreta, cuyas necesidades no siempre están bien cubiertas con un único método. El método de la superclase debe hacer dos cosas: definir el caso concreto que deberán observar todos sus descendientes, e implementar el caso concreto para la clase específica en la que aparece. Estos objetivos generalmente entran en conflicto. Por otra parte, cualquier superclase no abstracta puede separarse en una superclase abstracta y una subclase hoja concreta. La superclase abstracta contendrá todos los métodos que deben ser heredados por

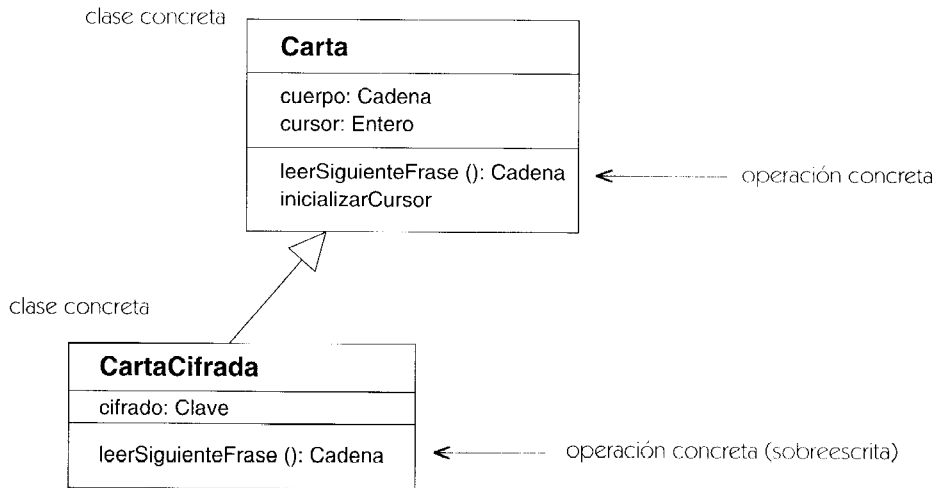


Figura 13.2 Una superclase concreta crea ambigüedad

las subclases, mientras que la subclase concreta contendrá los métodos necesarios para la clase instanciable específica. La regla de la superclase abstracta proporciona también una clara distinción entre una variable o parámetro que debe mantener el tipo concreto específico y una que puede tener el tipo de cualquiera de los descendientes de la superclase.

Considérese la declaración de la clase **Carta** de la Figura 13.2, la cual no sigue la regla de la superclase abstracta. Dicha clase tiene dos operaciones: **inicializarCursor** que pone el cursor al principio, y **leerSiguienteFrase** que devuelve el texto de la siguiente frase no leída. Sin embargo, la subclase **CartaCifrada** representa una carta cuyo texto ha sido cifrado y en la que la operación **leerSiguienteFrase** ha sido sobrescrita debido a que el texto debe ser descifrado antes de poder ser devuelto. La implementación de la operación es completamente diferente. El hecho de que **Carta** sea una superclase concreta no permite distinguir entre un parámetro que fuera una **Carta** ordinaria (no sobrescribible) y uno que pudiera ser tanto una **Carta** como una **CartaCifrada**.

El enfoque de la superclase abstracta permite distinguir la clase abstracta **Carta** (que puede ser tanto una carta cifrada como una no cifrada) y añadir la clase **CartaNoCifrada** para representar cada caso concreto, como se muestra en la Figura 13.3. En este caso, **leerSiguienteFrase** es una operación abstracta implementada por cada una de las subclases, mientras que **inicializarCursor** es una operación concreta, igual para las dos subclases. El modelo es simétrico.

Si se sigue la regla de la superclase abstracta, la declaración de clases abstractas puede determinarse automáticamente a partir de la jerarquía de clases, por lo que representarlo en diagramas es redundante.

Hay una excepción a la afirmación de que una clase hoja abstracta no sirve para nada: una clase abstracta declarada como espacio de nombres común de un conjunto de atributos y operaciones de ámbito global a la clase. Esta utilización es muy poco frecuente pero a veces es necesaria cuando se programa con lenguajes no orientados a objetos. Eso sí, debe evitarse siempre que sea posible, ya que las variables globales vulneran el espíritu del diseño orientado a objetos introduciendo dependencias globales. Una clase unitaria a menudo proporciona la misma funcionalidad de una manera más fácil de ampliar.

Véase [Gamma-95], Patrón Unitario (Singleton).

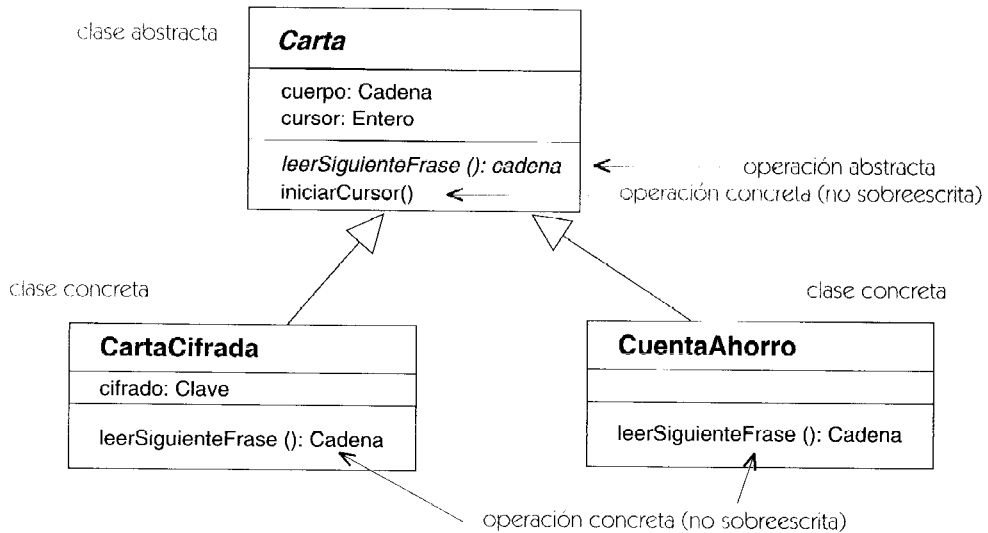


Figura 13.3 Una superclase abstracta elimina la ambigüedad

acceso

Dependencia de permiso que posibilita a un paquete hacer referencia a elementos de otro paquete.

Véase amigo/a, importar, visibilidad.

Semántica

Un paquete (cliente) que hace referencia a un elemento de otro paquete (proveedor) debe importar el paquete que contiene el elemento utilizando una dependencia «**access**» o «**import**» del paquete cliente hacia el paquete proveedor. Un paquete consigue implícitamente acceso a todos los paquetes importados por un paquete dentro del cual está anidado (esto es, los paquetes anidados pueden ver todo aquello que ven sus paquetes contenedores).

Un elemento de un paquete tiene acceso a todos los elementos que son *visibles dentro del paquete*. Las reglas de visibilidad se pueden resumir de la siguiente forma:

- Todo elemento definido en un paquete es visible dentro del mismo paquete.
- Si un elemento es visible dentro de un paquete, es visible en todos los paquetes anidados dentro del mismo.
- Si un paquete accede o importa otro paquete, todos los elementos definidos con visibilidad pública en el paquete accedido o importado son visibles dentro del paquete que los importa.
- Si un paquete es hijo de otro, los elementos definidos con visibilidad pública o privada en el paquete padre son visibles dentro del paquete hijo.

- Las dependencias de acceso e importación no son transitivas. Si A ve a B y B ve a C, no necesariamente A verá a C.

Una consecuencia de esto es que un paquete no puede ver el interior de sus propios paquetes anidados a menos que acceda a ellos y que los contenidos que haya dentro de dichos paquetes sean públicos. Algunas reglas más sobre visibilidad:

- Los contenidos de un clasificador, es decir, atributos, operaciones y clases anidadas, son visibles dentro del paquete si tienen visibilidad pública en el clasificador. Obsérvese que los contenidos no estructurados de un subsistema están gobernados por las reglas sobre visibilidad de los paquetes anteriormente establecidas, mientras que los atributos y operaciones se gobiernan por esta regla.
- Los contenidos de un clasificador son visibles dentro de un clasificador descendiente si tienen visibilidad pública o protegida en el clasificador.
- Todo lo que contiene un clasificador es visible para los elementos internos del clasificador, incluyendo métodos internos o máquinas de estados del clasificador.

El caso normal sencillo concierne a los elementos de paquetes que están al mismo nivel. En este caso, un elemento puede ver todos los elementos de su propio paquete y todos los elementos que tengan visibilidad pública de aquellos paquetes importados por su paquete. Una clase puede ver las características públicas de las clases que ve y también las características protegidas de sus ascendientes.

Notación

Una dependencia de acceso se representa mediante una flecha con línea discontinua que sale del paquete cliente y apunta al paquete proveedor. La flecha está etiquetada con la palabra clave **«access»**.

Discusión

La Figura 13.4 muestra un ejemplo de acceso entre dos paquetes que se encuentran al mismo nivel. El paquete P puede acceder al paquete Q, pero el paquete Q no puede acceder a P. Las clases K y L del paquete P pueden ver la clase pública M del paquete Q, pero no la clase privada N. Las clases M y N no pueden ver ninguna clase del paquete P, aunque la visibilidad de la clase K es pública, debido a que Q no tiene acceso a P. Para que una clase de un paquete Q sea visible en un paquete P que está al mismo nivel que Q, la clase debe tener visibilidad pública, y Q debe ser accedido o importado por el paquete P.

La Figura 13.5 muestra un caso de visibilidad y declaraciones de acceso algo más complejo. El símbolo que precede al nombre del elemento representa la visibilidad del mismo fuera de su propio contenedor: + para visibilidad pública, # para protegida (visible sólo en descendientes) y – para visibilidad privada (no es visible fuera del contenedor).

La clase A puede ver las clases C y E porque está dentro de los paquetes X e Y.

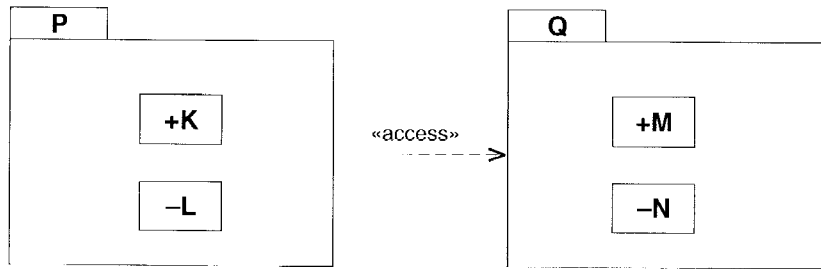


Figura 13.4 Acceso entre iguales

Las clases C y A pueden ver la clase D porque el paquete Y importa el paquete Z. La clase A está anidada dentro del paquete Y, por lo que puede ver todo aquello que ve Y.

Las clases A, C y E pueden ver a B porque están anidadas en el paquete X, que implementa el paquete V el cual contiene a B. Sin embargo, no pueden ver a F porque tiene visibilidad privada dentro de su propio paquete V. La clase F no es visible fuera de V.

La clase E no ve a D porque D está en el paquete Z, que no ha sido importado por el paquete X.

Ni la clase C ni la clase E pueden ver la clase A, debido a que A está en el paquete U, que no ha sido importado por ningún otro paquete.

Las clases B y F ven las clases D y E que están en paquetes que contienen al paquete V, al que ellas pertenecen. También pueden ver la clase C, que está en el paquete Y, pues dicho paquete es importado por el paquete X que engloba a V. El hecho de que F sea privada no limita el hecho de que pueda ver otras clases, pero evita que otras clases la vean a ella.

Las clases B y F se ven mutuamente porque están en el mismo paquete. La clase F es privada para las clases de otros paquetes, pero no para las que están en su mismo paquete.

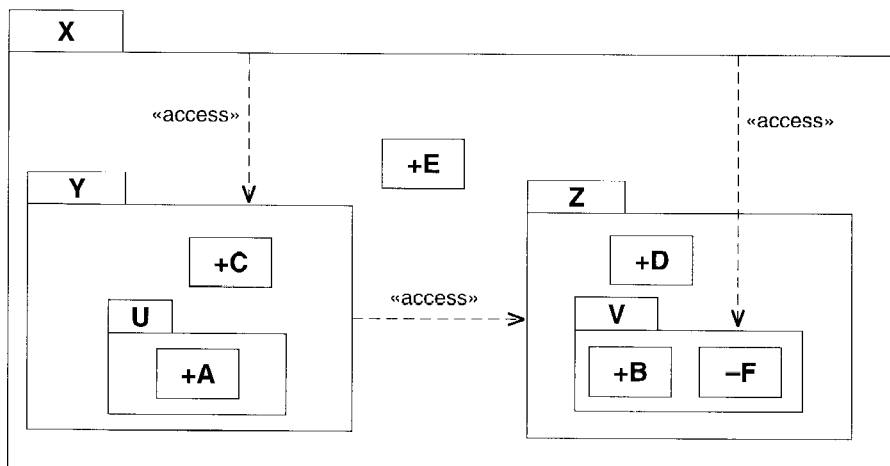


Figura 13.5 Reglas de acceso

acción

Computación atómica ejecutable que produce un cambio en el estado del modelo o que devuelve un valor. Contrastar con: actividad.

Véase también acción de entrada, acción de salida, transición.

Semántica

Una acción es una computación atómica, es decir, no puede ser finalizada por una orden de terminación externa. Puede asociarse a una transición en una máquina de estados (entre dos estados o bien dentro de un estado simple) o a un paso en una interacción. Generalmente es una operación primitiva o pseudo-primitiva sobre el estado de un sistema, a menudo sobre el estado de un objeto simple. Las acciones típicas son: asignación de un valor a un atributo, acceso a la información de atributos o enlaces, creación de nuevos objetos o enlaces, operaciones aritméticas sencillas y envío de señales a otros objetos. Las acciones son los pasos discretos con que se construye el comportamiento. Se supone que las acciones tienen que ser computaciones “rápidas” para no perjudicar el tiempo de respuesta del sistema. El sistema puede ejecutar varias acciones simultáneamente y hacer que compartan el tiempo entre ellas, pero su ejecución debe ser independiente.

Las acciones pueden estar asociadas a una transición, ejecutándose cuando la transición se dispara. También pueden aparecer como acciones de entrada o de salida de los estados. Hay acciones disparadas por transiciones que entran o dejan un estado. Todas las acciones son atómicas, esto es, se ejecutan completamente y sin interrupción de otras operaciones.

Una actividad es también una computación, pero puede tener una estructura interna y recibir una orden de terminación externa de una transición de un evento externo, y puede asociarse a un estado pero no a una transición. A diferencia de las acciones, las actividades pueden persistir indefinidamente hasta ser finalizadas externamente, aunque también pueden terminar por sí mismas. Una acción no puede ser finalizada externamente y debe estar asociada a una transición o a la entrada o salida de un estado, nunca a un estado en sí mismo.

Estructura

Una acción tiene un conjunto de objetos destino, una referencia a la señal a enviar o a la operación a realizar (a los que colectivamente se conoce como *solicitud*), una lista de valores para los argumentos y una expresión opcional de recurrencia que especifica la posible iteración.

Conjunto de objetos destino. Expresión de conjunto de objetos destino que produce un conjunto de objetos. En muchos casos, el conjunto contiene un objeto simple fijo. A todos los objetos del conjunto se les envía concurrentemente una copia del mensaje con la lista de argumentos correspondiente (es decir, se les comunica mediante difusión). Cada uno de ellos recibe y manipula independientemente una instancia distinta del mensaje. Si el conjunto está vacío no ocurre nada.

Solicitud. Denota la declaración de una operación o una señal. O bien se envía la señal a los objetos o bien se llama a la operación (en el caso de que la operación devuelva un valor, el conjunto debe contener un único objeto).

Lista de argumentos. Una lista de argumentos. Cuando se evalúa, los valores de la lista de argumentos deben corresponderse con los parámetros de la señal o de la operación. Los argumentos se proporcionan como parte de la llamada o del envío de la señal.

Recurrencia. Expresión de una iteración, que especifica cuántas veces hay que realizar la acción, y que opcionalmente puede incluir variables de iteración. Esta expresión también puede describir una acción condicional (una iteración con una o ninguna repetición).

Tipos de acciones

Acción de asignación. Una asignación es una acción que da a un atributo de un objeto un valor determinado. La acción tiene una expresión para el objeto destino (el nombre de un atributo del objeto), y una expresión para el valor a asignar al atributo del objeto.

Acción de llamada. Una acción de llamada provoca una invocación de una operación sobre un objeto: es una invocación a una operación de un objeto. La acción tiene un nombre de mensaje, una expresión para la lista de argumentos, y una expresión de conjunto de objetos destino. El destino puede ser un conjunto de objetos, en cuyo caso la llamada se produce concurrentemente y la operación no deberá por tanto devolver ningún valor. Si la operación devuelve un valor, el destino debe ser un único objeto.

Las acciones de llamada son asíncronas. El que realiza la llamada debe esperar a que se complete la operación invocada antes de recibir de nuevo el control. Si la operación se implementa como un evento de llamada, el que realiza la llamada espera hasta que el receptor ejecute la transición disparada por la llamada para retomar el control. Si la ejecución de la operación devuelve valores, el que realiza la llamada los recibe cuando retoma el control.

Acción de creación. Una acción de creación produce la instanciación y creación de un objeto (véase creación). La acción tiene una referencia a una clase, y opcionalmente una operación presente en la clase con una lista de argumentos. La ejecución de la acción crea una nueva instancia de la clase, cuyos atributos obtienen los valores correspondientes a la evaluación de las expresiones de valor iniciales. Si existe una operación de creación explícita, se ejecuta. La operación puede sobrescribir la inicialización de los valores de los atributos, normalmente utilizando valores como argumentos de la acción de creación.

Acción de destrucción. Una acción de destrucción produce la destrucción de un objeto destino. No existen otros argumentos. El resultado de ejecutar la acción es la destrucción del objeto, junto con todos sus enlaces y con todas las partes de las que estuviera compuesto (véase composición).

Acción de retorno. Una acción de retorno provoca la transferencia del flujo de objeto a aquél que llamó a la operación. Esta acción sólo está permitida dentro de una operación invocada por una llamada. La acción tiene una lista opcional de valores devueltos que se pone a disposición de quien realizó la llamada cuando se le devuelve el control. Si la acción adjunta fue invocada de forma asíncrona, el que llamó a la operación debe escoger explícitamente entre recibir el mensaje de retorno (como una señal), o perderlo.

Acción de envío. Una acción de envío crea una instancia de una señal y la inicializa con los argumentos obtenidos al evaluar las expresiones de argumento de la acción. La señal se envía a los objetos del conjunto de objetos destino obtenido evaluando la expresión destino de la

acción. Cada objeto del conjunto recibe su propia copia de la señal. El que envía la señal tiene su propio hilo de control y continúa ejecutándose, ya que el envío de una señal es asíncrono. La acción tiene un nombre de señal, una lista de expresiones para los argumentos de la señal y una expresión de conjunto de objetos destino para los objetos destino.

Si se omite el conjunto de objetos destino, la señal se envía a uno o más objetos determinados por la señal y por la configuración del sistema. Por ejemplo, una excepción se envía a un ámbito cerrado que viene determinado por las políticas del sistema.

Acción de terminación. Una acción de terminación provoca la destrucción del objeto propietario de la máquina de estados que contiene la acción (esto es, se suicida). La destrucción de un objeto es un evento al que pueden responder otros objetos.

Acción no interpretada. Una acción no interpretada es una estructura de control u otra estructura no definida en UML.

Notación

UML no tiene un lenguaje fijo de acciones. Se espera que los que realizan los modelos elegirán un lenguaje de programación real para escribir las acciones. La siguiente adaptación de OCL se utiliza en este libro para escribir pseudocódigo, pero no es parte del estándar.

Acción de asignación

destino := expresión

Acción de llamada

conjunto-objetos-destino . nombre-operación (argumento_{lista})

Acción de creación

new nombre-clase (argumento_{lista})

Acción de destrucción

objeto . destroy ()

Acción de retorno

return expresión_{lista}

Acción de envío

conjunto-objetos-destino . nombre-señal (argumento_{lista})

Acción de terminación

terminate

Acción no interpretada

if (expresión) **then** (acción) **else** (acción)

Si es necesario distinguir explícitamente entre llamada y envío, se pueden anteponer las palabras clave **call** o **send** a la expresión de forma opcional.

Discusión

La especificación de UML define un conjunto de acciones con la esperanza de que se añadirán otras en la implementación real de las herramientas que trabajen con UML. Esta decisión fue el resultado del compromiso entre el deseo de precisión y la necesidad de que los desarrolladores trabajaran con varios lenguajes, lo cual abarca un amplio espectro de conceptos semánticos. Existen muchas más variaciones en la semántica de ejecución entre lenguajes de programación de la que existe en las estructuras de datos o en el conjunto de estructuras de control disponibles. Las diferencias sutiles entre lenguajes son difíciles de plasmar de forma práctica, sin tener en cuenta si es posible hacerlo teóricamente. La selección de un lenguaje de programación como base para un lenguaje de acciones podría, sin embargo, tener el efecto de desanimar a los que utilizan otros lenguajes, lo cual no queríamos hacer. La semántica de acciones ha quedado pues algo incompleta y ambigua dentro del propio UML. Para precisar la semántica, hay que añadir a UML la semántica del lenguaje de acciones (a menudo un lenguaje estándar de programación) que se está utilizando. Algunos críticos se han quejado de que UML es impreciso debido a su libertad, pero lo es sólo hasta el grado de imprecisión del lenguaje de acción elegido. El auténtico defecto es que UML no impone una *lingua franca* de acciones y otras expresiones, lo cual es posible sólo a duras penas en el políglota mundo actual de la informática a pesar de las emotivas peticiones al respecto.

acción asíncrona

Solicitud en la cual el objeto emisor no hace una pausa para esperar los resultados. Envío.

Véase enviar, acción síncrona.

acción de entrada

Una acción realizada cuando se entra en un estado.

Véase también acción de salida, atómico/a, máquina de estados, transición.

Semántica

Un estado puede tener una acción de entrada opcional unida a él. Siempre que se entra en el estado se ejecuta la acción de entrada, después de las acciones asociadas a las transiciones entrantes o a la salida de los estados previos y antes de cualquier acción asociada a estados internos. La acción de entrada no se puede evadir por ningún medio. Está garantizado que se ha ejecutado siempre que el estado que la posee o un estado anidado dentro de él esté activo.

Orden de ejecución. En una transición entre dos estados con acciones de entrada y salida en las cuales la transición también tiene una acción, el orden de la ejecución es: Cualquier acción

de salida se ejecuta en el estado origen y los estados a los que engloba hacia fuera, pero no incluye a los estados que engloban tanto al estado de origen como al destino. Entonces se ejecuta la acción en la transición, después de lo cual se ejecutan las acciones de la entrada (las más externas primero) en los estados englobados dentro del estado, hacia abajo incluyendo el estado destino. La Figura 13.96 muestra algunas transiciones con acciones múltiples.

Notación

Una acción de entrada se codifica usando la sintaxis para una transición interna con el nombre de la **entrada** del evento simulado (que es por lo tanto una palabra reservada y no se puede utilizar como nombre real del evento).

entrada / secuencia-acción o *entry* / secuencia-acción

Sólo se puede unir una acción de entrada a un estado, pero la acción puede ser una secuencia, de modo que no se pierde generalidad.

Discusión

Las acciones de entrada y salida no son esenciales semánticamente (la acción de entrada se podría unir a todas las transiciones entrantes) pero facilitan la encapsulación de un estado para poder separar el uso externo de él de su construcción interna. Permiten definir acciones de inicialización y de finalización, sin la preocupación de que puedan evitarse. Son particularmente útiles con excepciones, porque definen las acciones que deben ser realizadas incluso si ocurre una excepción.

Una acción de entrada es útil para realizar una inicialización que deba hacerse cuando se entra al estado por primera vez. Un uso es inicializar las variables que capturan información durante un estado.

Por ejemplo, una interfaz de usuario para permitir la entrada, por un teclado numérico, de un número de teléfono o de un número de cuenta, limpiaría el número de la entrada. El reajuste de un contador de error, tal como el número de fallos en la contraseña, es otro ejemplo. La asignación de almacenamiento temporal, necesario durante el estado, es otro uso para una acción de entrada.

A menudo se utilizan juntas una acción de entrada y una acción de salida. La acción de entrada asigna recursos, y la acción de salida los libera. Incluso si ocurre una transición externa, se liberan los recursos. Esto es una buena manera de manejar errores de usuario y excepciones. Los errores de usuario accionan transiciones de alto nivel que abortan estados anidados, pero los estados anidados tienen una oportunidad de limpiar antes de perder el flujo de control.

acción de salida

Una acción que se realiza cuando se sale de un estado.

Véase también acción de entrada, atómico/a, máquina de estados, transición.

Semántica

Un estado puede tener asociada una acción de salida. Se ejecuta siempre que se salga del estado de cualquier manera, después de cualquier otra acción asociada a estados o transiciones más internos y antes de cualquier acción unida a estados más externos. La acción de salida no se puede evadir por ningún medio. Está garantizado que se ejecutará antes de que el flujo de objeto abandone el estado que la posee.

Las acciones de entrada y de salida no son esenciales semánticamente (la acción de salida se podría unir a todas las transiciones salientes), pero facilitan la encapsulación de un estado para poder separar el uso externo de él de su construcción interna. Permiten definir acciones de inicialización y de terminación, sin la preocupación de que puedan ser evitadas. Son particularmente útiles con excepciones, porque definen las acciones a realizar incluso si ocurre una excepción.

Notación

Una acción de salida se codifica usando la sintaxis para una transición interna con el evento simulado llamado **salida** (que es, por lo tanto, una palabra reservada y no se puede utilizar como nombre real del evento).

salida / secuencia-acción o exit / secuencia-acción

A un estado solamente se puede asociar una acción de salida, pero la acción puede ser una secuencia de acciones, así que no se pierde ninguna generalidad.

Discusión

Una acción de salida es útil para realizar una limpieza general cuando se sale de un estado. El uso más significativo de las acciones de salida es liberar el almacenamiento temporal y otros recursos asignados durante la ejecución del estado (generalmente, un estado con el detalle anidado).

A menudo se utilizan juntas una acción de entrada y una acción de salida. La acción de entrada asigna recursos, y la acción de salida los libera. Los recursos son liberados incluso si ocurre una excepción.

acción síncrona

Solicitud en la cual el objeto emisor se detiene a esperar una respuesta; también es una llamada. Contrastar con: acción asíncrona.

activación

Ejecución de una operación. Una activación (también conocida como foco de control) representa el período durante el cual un objeto realiza una operación, bien directamente o bien a

través de una operación subordinada. Modela tanto la duración de la ejecución como la relación de control entre la ejecución y los que hicieron la llamada a la misma. En un computador y lenguaje de programación convencionales, una activación se correspondería con un bloque de llamada en la pila.

Véase llamada, diagrama de secuencia.

Semántica

Una activación es una instancia de ejecución de una operación, que incluye el período durante el cual la operación llama a otras operaciones subordinadas (véase llamada). Su contexto comprende un conjunto de variables locales accesibles únicamente por la activación, una posición actual dentro del método (u otra descripción de comportamiento), y una referencia (la *referencia de retorno*) a la activación que representa el contexto de llamada, que retoma el control cuando termina la activación actual. Una activación sin referencia de retorno debe ser el resultado de una transición sobre la máquina de estados de la clase de un objeto activo; cuando se completa la activación, la máquina de estados simplemente espera el próximo evento.

Obsérvese que esta definición describe un procedimiento ordinario, tal y como se implementa en una típica máquina de von Neumann. Pero expresarlo de forma general significa aplicarlo también a un entorno distribuido, en el cual no hay memoria compartida y en el que el marco de pila está formado por una lista enlazada de activaciones en diferentes espacios de memoria.

Notación

Una activación se representa en un diagrama de secuencia utilizando un rectángulo alto y delgado (una barra vertical hueca) cuya parte superior está alineada con el momento de su iniciación y cuya parte inferior lo está con el momento de su finalización. La operación a realizar se representa mediante una etiqueta de texto al lado del símbolo de activación o en el margen izquierdo, dependiendo del estilo.

De forma alternativa, también es posible que el símbolo de mensaje entrante indique la operación, en cuyo caso se puede omitir la etiqueta en la propia activación. Si el flujo de objeto es de procedimientos, entonces el extremo superior del símbolo de activación estará en la punta de la flecha del mensaje entrante que inicia la acción, y del extremo inferior saldrá la flecha del mensaje de retorno.

Si hay actividad concurrente entre varios objetos, la activación muestra la ejecución de un objeto concurrente. A menos que los objetos se comuniquen, las actividades concurrentes son independientes, y no es relevante reflejar sus tiempos relativos de ejecución.

Cuando se trata de código de procedimientos, una activación muestra o bien la duración durante la cual un procedimiento está activo en el objeto o bien el tiempo durante el que está activo un procedimiento subordinado llamado por el procedimiento original, posiblemente en algún otro objeto. En otras palabras, se muestran simultáneamente todas las activaciones de procedimientos anidados. Este conjunto de activaciones anidadas simultáneas forma el marco de pila de la computación en un computador convencional. En caso de una segunda llamada a un objeto con una activación existente, el segundo símbolo de activación se dibuja ligeramente a la derecha del primero, de forma que den la sensación de estar apilados. Las llamadas

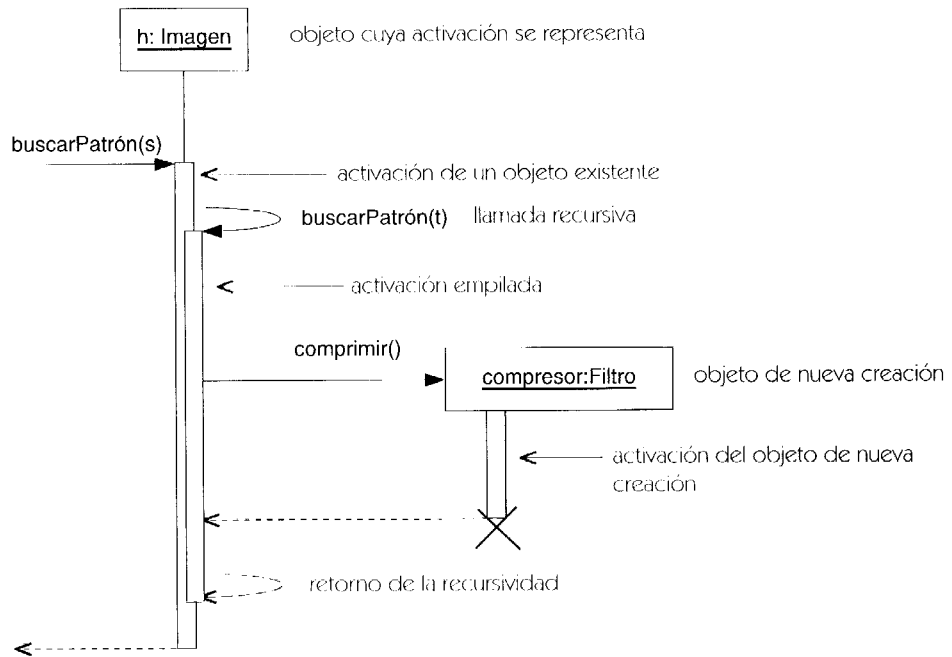


Figura 13.6 Activaciones

apiladas pueden tener cualquier nivel de anidamiento. Las llamadas pueden ser a la misma operación (llamada recursiva) o a diferentes operaciones del mismo objeto.

Ejemplo

La Figura 13.6 muestra las activaciones producidas a partir de las correspondientes llamadas, incluyendo una llamada recursiva.

actividad

Ejecución no atómica en curso dentro de una máquina de estados. Contrastar con: acción.

Véase también transición de finalización, estado.

Semántica

Una actividad es la ejecución de una subestructura dentro de una máquina de estados, pero de una subestructura que tiene una duración y posibles puntos de interrupción. Cualquier transición que fuerce la salida de la región de control aborta la actividad. Una actividad no termina por el hecho de que se dispare una transición interna, porque no hay cambio de estado, pero la acción asociada a la transición interna sí podría terminarla explícitamente.

Una actividad se puede modelar mediante estados anidados, utilizando una referencia a una submáquina o empleando una expresión de actividad.

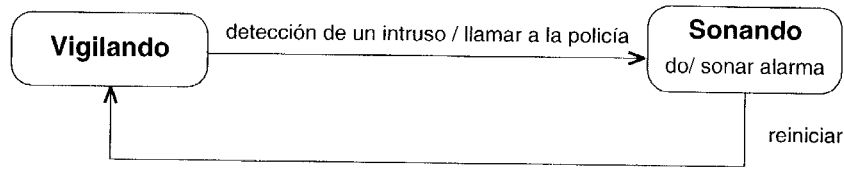


Figura 13.7 Acción y actividad

Ejemplo

La Figura 13.7 muestra un sistema de alarma que ilustra la diferencia entre una acción y una actividad. Cuando se produce el evento **detección de un intruso**, el sistema dispara una transición. Como parte de la transición, se lleva a cabo la acción **llamar a la policía**, que es una acción puesto que es algo atómico (y normalmente rápido). Mientras se ejecuta la acción no se acepta ningún evento. Después de realizar la acción el sistema pasa al estado **Sonando**, y mientras está en este estado realiza la actividad **sonar alarma**. Una actividad tarda un tiempo en realizarse, tiempo durante el cual la actividad puede ser interrumpida por algún evento. En este caso, la actividad **sonar alarma** no termina por sí sola, ya que continúa ejecutándose mientras el sistema se encuentre en el estado **Sonando**. Cuando se produce el evento **reiniciar**, se dispara la transición que devuelve al sistema al estado **Vigilando**.

Cuando el estado **Sonando** deja de estar activo, su actividad **sonar alarma** termina.

activo

Estado en el que se ha entrado y del que aún no se ha salido; estado que mantiene un objeto.

Véase también clase activa, objeto activo.

Semántica

Un estado pasa a estar activo cuando se dispara una transición de entrada, y deja de serlo cuando se dispara una transición de salida. Si un objeto tiene un hilo de control, entonces al menos hay un estado activo (en un caso extremo, una clase puede tener sólo un estado, lo que supone que la respuesta ante un evento es siempre la misma). Si un estado está activo dentro de la máquina de estados de la clase de un objeto, se dice que el objeto *mantiene* el estado, o que *está* en ese estado.

Un objeto puede tener más de un estado simultáneamente. El conjunto de estados activos se denomina configuración del estado activo. Si un estado anidado está activo, todos los estados que lo contienen están también activos. Si el objeto permite concurrencia, puede haber más de un subestado activo al mismo tiempo. Cada transición afecta, al menos, a unos pocos estados de la configuración del estado activo. Ante una transición, los estados activos no afectados por la misma permanecen activos.

Un estado compuesto puede ser secuencial o concurrente. Si es secuencial y está activo, entonces habrá exactamente uno de sus subestados activo. Si es concurrente y está activo, en-

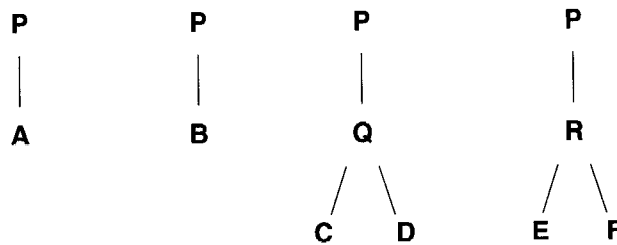
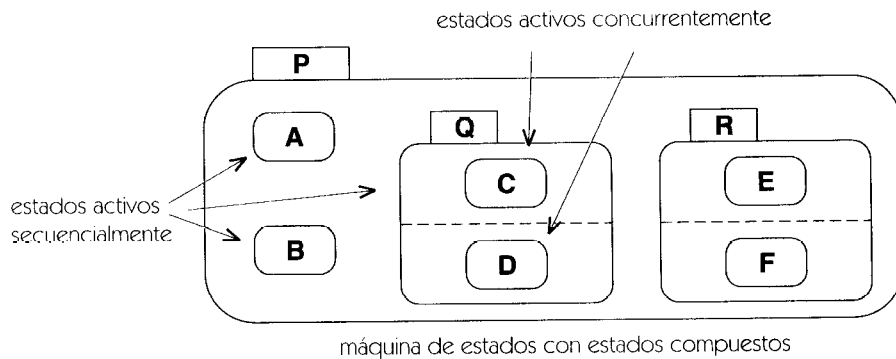
tonces todos los subestados que contiene estarán activos. En otras palabras, un estado compuesto se expande en un árbol Y-O de subestados activos, de forma que en cada nivel hay siempre estados activos.

Una transición que atraviesa en la frontera de un estado compuesto debe estructurarse de forma que mantenga las restricciones de concurrencia. Una transición en un estado secuencial compuesto normalmente tiene un estado origen y otro destino, y disparar una transición de este tipo no cambia el número de estados activos. Sin embargo, una transición en un estado concurrente compuesto tendrá un estado origen y uno destino para cada una de las regiones en que se divide el estado concurrente compuesto, y una transición de este tipo se denomina división. Si se omiten una o más regiones como destino, el estado inicial de cada región omitida está presente de forma implícita como destino, pero si una de las regiones carece de estado inicial, entonces se dice que el modelo está mal modelado. El disparar una transición de este tipo incrementa el número de estados activos, situación que se invierte a la salida del estado concurrente compuesto.

Véase máquina de estados, donde se lleva a cabo una completa discusión sobre la semántica de los estados concurrentes y las transiciones complejas.

Ejemplo

La parte superior de la Figura 13.8 muestra un ejemplo de máquina de estados que tiene estados compuestos, tanto secuenciales como concurrentes. Se han omitido las transiciones para



posibles configuraciones del estado activo

Figura 13.8 Estados activos concurrentemente

centrar la atención en los estados. La parte inferior de la figura muestra las diversas configuraciones de estados que pueden estar activas concurrentemente.

En este ejemplo hay cuatro posibles configuraciones de estados activos. Sólo los estados hoja son concretos, ya que los estados de más alto nivel son abstractos, es decir, un objeto no puede estar en uno de ellos sin estar también en uno de sus estados hoja anidados. Por ejemplo, un objeto no puede estar en el estado Q sin estar en alguno de los subestados de Q. Como Q es concurrente, tanto C como D tienen que estar activos si Q lo está. Cada estado hoja se corresponde con un hilo de control. En un ejemplo más amplio, el número de posibles configuraciones crecería exponencialmente y sería imposible mostrarlas todas, de aquí la ventaja de la notación.

actor

Abstracción de las entidades externas a un sistema, subsistemas o clases que interactúan directamente con el sistema. Un actor participa en un caso de uso o conjunto coherente de casos de uso para llevar a cabo un propósito global.

Véase también caso de uso.

Semántica

Un actor abstrae y caracteriza a un usuario externo o a un conjunto de usuarios externos relacionados que interactúan con el sistema o clasificador. Es una idealización con un propósito y significado concretos, que puede no corresponderse con objetos físicos. Un objeto físico puede combinar propósitos dispares y por tanto ser modelado por varios actores y viceversa, diferentes objetos físicos podrían incluir el mismo propósito, y por tanto ser modelados como el mismo actor. El objeto usuario puede ser una persona, un sistema informático, otro subsistema u otro tipo de objeto. Por ejemplo, el conjunto de actores de un sistema de redes de computadores podría incluir los actores **Operador**, **Administrador del Sistema**, **Administrador de Bases de Datos** y **Usuario** sin más. También podría haber actores no humanos como por ejemplo **Cliente Remoto**, **Reloj Maestro** o **Impresora de Red**.

Cada actor define un conjunto de roles que los usuarios de un sistema asumen cuando interactúan con el mismo. El conjunto completo de actores describe todas las diferentes formas de comunicación entre los usuarios externos y el sistema. Cuando se implementa el sistema, los actores se implementan como objetos físicos. Un objeto físico puede implementar más de un actor si puede llevar a cabo todos sus roles. Por ejemplo, una persona puede ser tanto vendedor como cliente de una misma tienda; estos actores no están inherentemente relacionados, pero ambos pueden ser implementados por una persona. Cuando se diseña un sistema, los diferentes actores se realizan mediante clases de diseño (véase realización).

Las diferentes interacciones de los actores con un sistema se cuantifican en casos de uso. Un caso de uso es una pieza coherente de funcionalidad que implica a un sistema y a sus actores para llevar a cabo algo que tiene sentido para los actores. Un caso de uso puede implicar a uno o más actores, de la misma forma que un mismo actor puede participar en más de un caso de uso. Los actores vienen determinados por los casos de uso que realizan y por los roles que

desempeñan en los diferentes casos de uso. Un actor que no participa en ningún caso de uso carece de sentido.

Un modelo de casos de uso caracteriza los tipos de comportamiento que proporciona una entidad, como por ejemplo un sistema, un subsistema o una clase, en sus interacciones con entidades externas. En el caso de un sistema, los actores pueden ser realizados tanto por usuarios humanos como por otros subsistemas. Cuando se trata de un subsistema o de una clase, los elementos externos pueden ser actores del sistema global o pueden ser otros elementos internos del sistema, como otros subsistemas u otras clases.

Las instancias de actores se comunican con el sistema mediante el envío y recepción de instancias de mensajes (señales y llamadas) desde y hacia instancias de casos de uso, y en el nivel de realización, desde y hacia los objetos que implementan el caso de uso. Todo lo anterior se expresa mediante asociaciones entre el actor y el caso de uso.

Un actor puede listar el conjunto de señales que envía y que recibe, y mantener una lista de las interfaces que ofrece y que requiere. Las interfaces de un actor deben ser compatibles con las de los casos de uso con los que se comunica. En otras palabras, un actor debe recibir todas las señales que un caso de uso sea capaz de enviarle y no debe mandarle al caso de uso señales que éste no sea capaz de recibir. Las interfaces de un actor restringen las clases con las que pueda corresponderse (que puedan implementar su comportamiento). Un actor puede también tener una lista de atributos que caracterice su estado.

Generalización

Pueden existir semejanzas entre dos o más actores; esto es, pueden comunicarse con el mismo conjunto de casos de uso de la misma manera. Esta semejanza se expresa mediante la generalización en otro actor, posiblemente abstracto, que modele los aspectos comunes. Los actores descendientes heredarán los roles y relaciones con casos de uso del actor antecesor. Una instancia de un actor descendiente siempre se puede utilizar en aquellos casos en los que se espera una instancia del actor antecesor (principio de capacidad de sustitución). Un descendiente incluye los atributos y operaciones de sus antecesores.

Notación

Un actor se puede representar mediante un símbolo de clase (rectángulo) con el estereotipo «actor». El icono estándar de un actor es el “monigote”, con el nombre del actor debajo. El actor puede tener compartimentos que muestren los atributos y eventos que recibe, y también dependencias que muestren los eventos que envía, como cualquier clasificador normal (Figura 13.9).



Figura 13.9 Símbolo de actor

agregación

Forma de asociación que especifica una relación todo-parte entre un agregado (el todo) y las partes que lo componen.

Véase también composición.

Semántica

Una *asociación binaria* puede declararse como agregación, o lo que es lo mismo, como relación todo-parte. Un extremo de la asociación se designa como agregado mientras que el otro no se marca de forma especial. No pueden ser agregados (o compuestos) ambos extremos, pero sí que pueden estar los dos sin marcar, en cuyo caso no se trataría de una agregación.

Los enlaces instanciados de las asociaciones de agregación obedecen ciertas reglas. La relación de agregación es transitiva y antisimétrica a través de los enlaces de la agregación, incluso a través de aquellos que pertenecen a diferentes relaciones de agregación. La propiedad transitiva significa que tiene sentido decir que “B es parte de A” si existe una ruta de enlaces de agregación desde B hasta A en sentido directo (en este ejemplo, de las partes hacia el todo). La antisimetría obliga a que no existan bucles en las rutas dirigidas de los enlaces de agregación. Esto es, no es posible que ningún objeto sea parte de sí mismo directa o indirectamente. Uniendo los dos roles, el grafo de enlaces de agregación de todas las asociaciones de agregación es un grafo parcialmente ordenado, un grafo sin bucles (el caso más común de orden parcial sería un árbol). En la Figura 13.10 se muestra un ejemplo.

Una ruta dirigida de enlaces desde el objeto B hasta el objeto A implica que existe una ruta dirigida de asociaciones de agregación de la clase B a la clase A, pero la ruta de asociaciones puede implicar bucles en los que la misma clase aparezca más de una vez. Una ruta dirigida de asociaciones de agregación de una clase a sí misma es una *recursividad*.

Existe una relación más fuerte de agregación llamada composición. Un compuesto es un agregado con las restricciones adicionales de que un objeto puede ser parte sólo de un compuesto, y de que el objeto compuesto tiene la responsabilidad de la disponibilidad de sus partes a la hora de crearlas y destruirlas.

Véase composición para más detalles.

En la agregación simple, una parte puede pertenecer a más de un agregado, y lo que es más, puede existir independientemente del agregado. A menudo el agregado “necesita” las partes, en el sentido de que puede ser considerado como una colección de partes, pero las partes pueden existir por sí mismas, sin tener que ser vistas como partes únicamente. Por ejemplo, una ruta es poco más que un conjunto de segmentos, pero un segmento puede existir independientemente de si pertenece o no a una ruta, y además el mismo segmento puede pertenecer a más de una ruta.

Véase asociación y extremo de asociación para más información sobre las propiedades de la agregación.

Notación

Una agregación se representa mediante un rombo o diamante hueco en el extremo de la línea de la asociación que se conecta a la clase agregada (véase Figura 13.11). Si la agregación es una composición, su representación es un rombo relleno (véase Figura 13.54). No pueden tener indicadores de agregación ambos extremos.

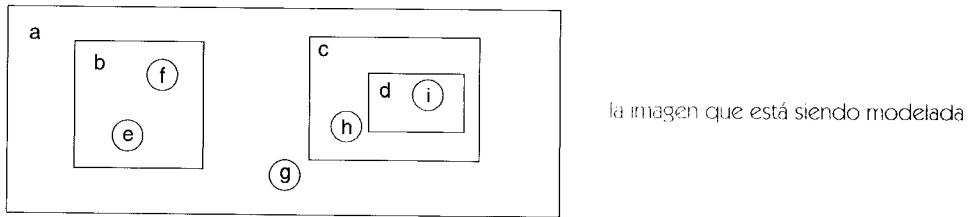


diagrama de clases: recursividad empleando agregación

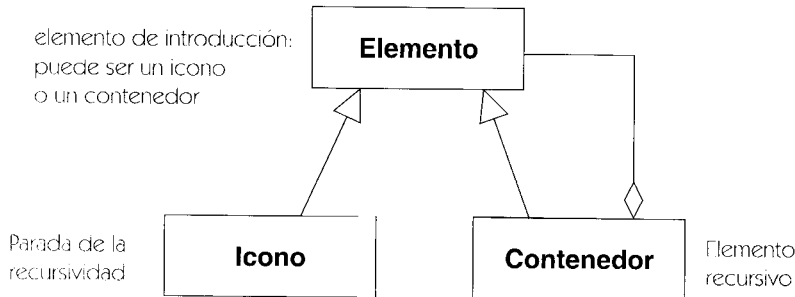


diagrama de objetos: no hay bucles entre los objetos

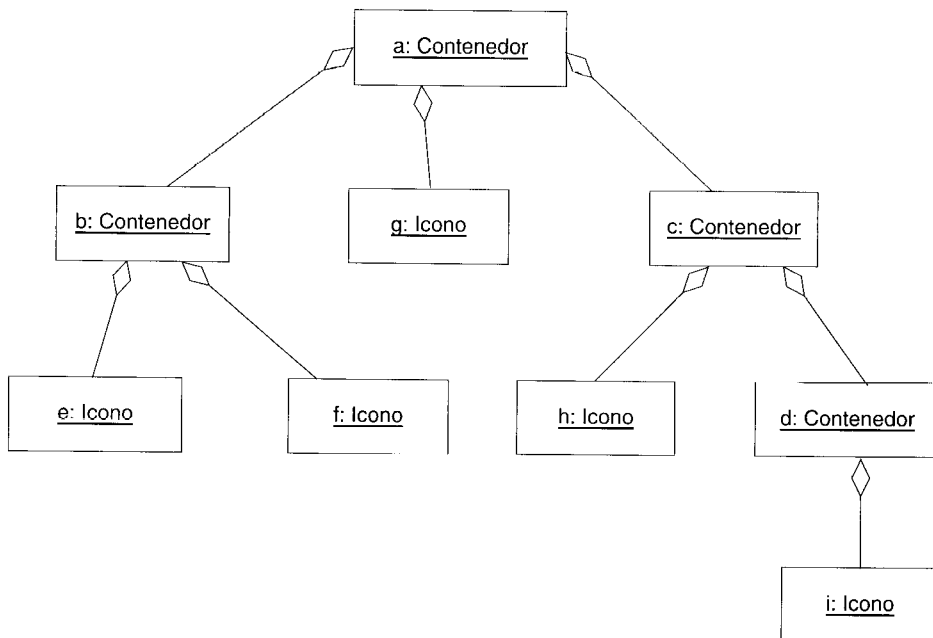


Figura 13.10 Las agregaciones de objetos son acíclicas

Una clase agregada tiene múltiples partes, pero cada relación entre la clase agregada y cada una de las partes es una asociación distinta (véase Figura 13.12).

Si hay dos o más asociaciones de agregación para la misma clase agregada, se pueden representar como un árbol combinando los extremos de la agregación en un único segmento

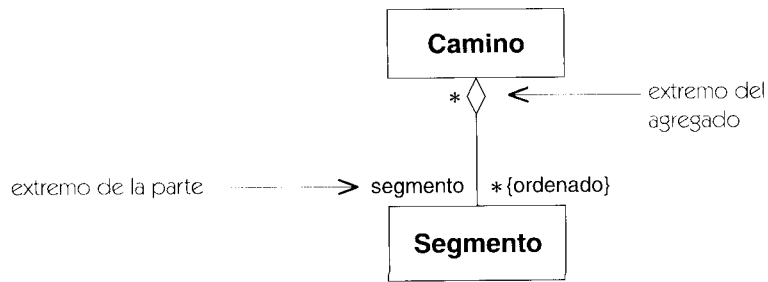


Figura 13.11 Notación de agregación

(véase Figura 13.13), pero esto requiere que todos los adornos de los extremos de la agregación sean consistentes; por ejemplo, deben tener la misma multiplicidad. Dibujarlo mediante un árbol es una mera cuestión de presentación que no aporta ninguna semántica adicional.

Discusión

La distinción entre asociación y agregación es muy a menudo un asunto de gustos más que una diferencia semántica real. Hay que recordar que la agregación es una asociación. La agregación conlleva la idea de que el agregado es la suma de sus partes, y en efecto, la única semántica adicional que aporta a la asociación es la restricción que impide que en los enlaces de agregación existan bucles, que es importante conocer. Otras restricciones, como la existencia de dependencia, las expresa la multiplicidad, no la agregación. A pesar del poco significado que

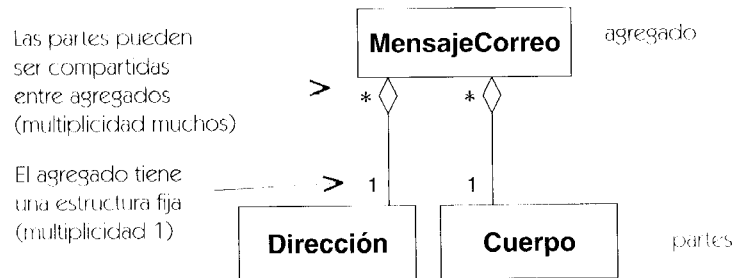


Figura 13.12 Un agregado con varias partes

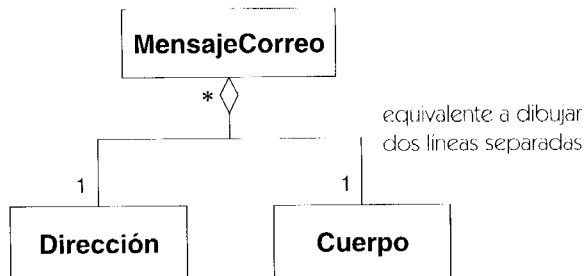


Figura 13.13 Representación en árbol para múltiples agregaciones de la misma clase

aporta, todo el mundo está convencido de que es necesaria (por diferentes razones). Piense en ello como en un placebo de modelado.

Hay algunas propiedades secundarias conectadas con la agregación, pero no son suficientes para requerir formar parte de la definición. Algunas de ellas son la propagación de operaciones desde el agregado a las partes (por ejemplo una operación mover), o la asignación de memoria compacta (de forma que el agregado y sus partes puedan cargarse eficientemente con una única transferencia de memoria). Algunos autores distinguen varios tipos de agregación, pero dichas distinciones son tan sutiles que probablemente sean innecesarias para un modelado común.

La agregación es una propiedad que trasciende una asociación particular. Se pueden componer agregaciones sobre diferentes pares de clases, y el resultado es una agregación. La agregación impone la restricción sobre las instancias de todas las asociaciones de agregación (incluyendo las asociaciones de composición) de que no pueden existir bucles entre los enlaces de agregación, incluyendo los enlaces de asociaciones diferentes. En cierto sentido, la agregación es una especie de generalización de la asociación en la cual las restricciones y algunas operaciones se aplican a asociaciones de muchos tipos específicos.

La composición tiene una semántica más específica, como el contenido físico y algunas nociones de propiedad. La composición resulta apropiada cuando cada parte es propiedad de un objeto y cuando las partes no tienen vida independiente separadas de su propietario. Es más útil cuando las partes tienen que ser asignadas e inicializadas en el momento en que se crea su propietario y no sobreviven a la destrucción del mismo. Los atributos de una clase tienen estas propiedades y pueden por ello ser considerados una forma de composición, aunque no se modelan explícitamente como tales. Utilizando la composición, se puede evitar la sobrecarga del gestor de memoria y el peligro de los punteros sueltos u objetos huérfanos. También resulta apropiado para situaciones en las cuales se han aislado una gran cantidad de atributos en una clase distinta por motivos de encapsulamiento y manipulación, pero los atributos realmente pertenecen a la clase principal. Las clases contenedoras utilizadas para implementar asociaciones son también candidatos obvios para partes compuestas, aunque por lo general, deberían ser generadas por un generador de código y no modeladas explícitamente. Obsérvese que una parte compuesta, como por ejemplo una clase contenedora, puede contener referencias (punteros) a partes no compuestas, pero los objetos referenciados no se destruyen cuando se destruye el objeto que los referencia.

agregación compuesta

Véase composición.

agregado

Clase que representa el todo en una asociación de agregación (todo-parte).

alcance

Extensión de un miembro de clasificador, tal como un atributo, operación o nombre de rol; esto es, si representa un valor en cada instancia o bien un valor que comparten todas las instancias

de un clasificador. Cuando se utiliza en solitario sin calificación alguna, indica un alcance de propietario.

Véase creación, alcance de propietario, alcance de destino.

Semántica

El alcance puede ser o bien de propietario o bien de destino.

Alcance de propietario. Indicación de si existe o no una ranura de atributo para cada instancia de una clase, o de si hay una sola ranura con el nombre dado para toda la clase.

instancia	Cada instancia del clasificador posee su propia copia independiente de una ranura de atributo o bien su propio conjunto de objetos asociados. Los valores de una ranura son independientes de los valores que haya en las otras ranuras. Ésta es la situación por defecto. Para una operación, la operación se le aplica a un objeto individual (en una operación normal).
clase	El clasificador en sí posee una copia de la ranura de atributo o del conjunto de objetos asociados. Todas las instancias del clasificador comparten el acceso a una sola ranura. Para una operación, la operación es aplicable a toda la clase, tal como una operación de creación o una operación que proporcione estadísticas relativas a todo un conjunto de instancias. Esta operación no se le puede aplicar a una instancia.

Alcance de destino. Una opción que indica si los valores de un atributo o los valores del destino de una asociación son o no instancias (por defecto) o clasificadores.

instancia	Cada ranura de atributo o cada enlace de la asociación contiene una referencia de una instancia del clasificador destino. El número de enlaces se ve limitado por la multiplicidad. Ésta es la situación por defecto.
clase	Toda ranura de atributo o todo enlace de la asociación contiene una referencia de la clase destino en sí. La información de la asociación queda por tanto determinada en tiempo de modelado y no cambia durante el tiempo de ejecución; de hecho no es necesario almacenarla en todos los objetos. Los enlaces implican a la clase en sí, no a sus instancias. Esto puede ser útil para cierta información de implementación, pero para la mayoría de los propósitos de modelado esta capacidad se puede ignorar.

Discusión

Los atributos o asociaciones de alcance de clase proporcionan valores globales para toda una clase y por tanto deberían utilizarse con cuidado o evitarse por completo, aunque los proporcionan casi todos los lenguajes de programación orientados a objetos. El problema es que implican una información global, lo cual viola el espíritu del diseño orientado a objetos. Además, la información global se vuelve problemática en sistemas distribuidos, porque im-